



Université de Bretagne Occidentale

Master 2 Informatique / Logiciels pour Systèmes Embarqués

Diagnostic et pronostic de défaillances dans les équipements rotatifs de l'Industrie 4.0

*Une comparaison entre autoencodeurs, Random Forest et une
architecture combinatoire sur microcontrôleur ESP32*

Présenté par :
Alexandre COUTÉ

Sous la direction de :
Catherine DEZAN, David ESPES & Isadora FERRÃO (UBO | Lab-STICC)
Ana Fernanda QUARESMA & Kalinka BRANCO (Universidade de São Paulo)

Année Universitaire 2025-2026

28 Juillet 2026

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude envers mes encadrantes à l'Université de Bretagne Occidentale (UBO), Catherine Dezan et Isadora Ferrão, pour leur suivi rigoureux, leurs conseils avisés et leur soutien constant tout au long de ce stage au sein du Lab-STICC. Leurs expertises respectives dans le domaine des systèmes embarqués et de l'intelligence artificielle m'ont permis de structurer ma démarche scientifique et de surmonter les nombreux défis techniques rencontrés au cours de ces travaux de recherche.

Je remercie également très chaleureusement mes encadrantes à l'Universidade de São Paulo (USP), Ana Fernanda Quaresma et Kalinka Branco. Ce projet m'a offert la formidable opportunité d'effectuer une mobilité de deux mois à São Carlos au Brésil (d'avril à juin), une expérience humaine, culturelle et scientifique extrêmement enrichissante. L'accueil qui m'a été réservé au sein de leur laboratoire et les échanges fructueux que nous avons pu avoir ont été un véritable moteur pour la réussite de ce stage et ont grandement contribué à l'aboutissement de l'architecture logicielle présentée dans ce document.

Résumé

Dans le contexte hautement concurrentiel de l'Industrie 4.0, l'arrêt inattendu d'une chaîne de production dû à des défaillances matérielles imprévues représente un coût économique et logistique critique pour les entreprises.

Ce rapport détaille de manière exhaustive la conception, l'optimisation et l'implémentation de différentes solutions d'intelligence artificielle embarquée (Edge Computing / TinyML) dédiées au diagnostic précoce de défaillances de roulements à billes par l'analyse de leurs signaux vibratoires. Afin de garantir une latence de traitement minimale, une indépendance vis-à-vis des réseaux externes et la sécurité absolue des données industrielles, les algorithmes développés sont déployés localement sur un microcontrôleur ESP32, un composant très contraint en ressources calculatoires et mémorielles. L'étude compare rigoureusement des approches non supervisées (Autoencodeur classique, Autoencodeur Variationnel) qui modélisent le comportement sain de la machine, et des approches supervisées (Random Forest) capables de catégoriser précisément les pannes connues.

Face au risque majeur de classification forcée inhérent aux modèles supervisés confrontés à des données inédites, nous proposons le développement d'une architecture algorithmique en deux étages alliant un modèle d'Autoencodeur et un modèle Random Forest. Dans cette hypothèse, l'Autoencodeur agit comme un filtre universel et intransigeant pour détecter toute anomalie, même totalement inconnue du système, tandis que le Random Forest est sollicité pour qualifier la panne spécifique si l'anomalie est avérée mais aussi pour corriger une potentielle erreur de l'Autoencodeur. Évaluées de manière approfondie sur les jeux de données reconnus CWRU et MaFaulDa, incluant des scénarios de tests extrêmes, ces solutions ont toutes respecté les contraintes drastiques de mémoire et de calcul de la cible embarquée. Leurs performances de précision et leurs temps d'inférence ont été comparés et analysés en détail, démontrant la viabilité de l'Edge AI pour la maintenance industrielle critique.

Table des matières

Remerciements	1
Résumé / Abstract	2
Table des figures	5
Liste des tableaux	6
1 Introduction	7
1.1 Contexte et Enjeux de l'Industrie 4.0	7
1.2 Concepts Clés : Analyse Vibratoire, Edge AI et TinyML	7
1.3 Cadre du Stage et Collaboration de Recherche	9
1.4 Objectifs du Stage et Plan du Rapport	10
2 Contexte Scientifique et IA	11
2.1 Mécanique des Défaillances : Le Roulement à Billes	11
2.2 Modèles d'Apprentissage Automatique Étudiés	12
2.2.1 L'approche Supervisée : Random Forest (RF)	12
2.2.2 L'approche Non-Supervisée : Autoencodeurs (AE et VAE)	13
3 Méthodologie et Déploiement Embarqué (TinyML)	16
3.1 Acquisition et Prétraitement des Données	16
3.2 Architecture Combinatoire Proposée	17
3.3 Contraintes Matérielles : La cible ESP32	18
3.4 Optimisation et Transpilation (TinyML)	19
3.4.1 Compression de l'Autoencodeur (TFLite Micro)	19
3.4.2 Transpilation native du Random Forest	20
3.5 Défis Techniques et Difficultés Rencontrées	21
3.6 Disponibilité et Utilisation du Code Source	22
4 Évaluation Expérimentale et Résultats	24
4.1 Analyse de la Séparation Spatiale et Fréquentielle	24
4.2 Évaluation sur les Classes Nominales (Matrices de confusion)	25
4.3 Évaluation sur les Cas Extrêmes et Pièges	26
4.4 Performances et Empreinte Matérielle	28
4.4.1 Résultats	28
4.4.2 Performances théoriques	29

5 Conclusion et Perspectives	31
5.1 Bilan des travaux	31
5.2 Valorisation scientifique	32
5.3 Perspectives d'évolution	33
Bibliographie	34
A Article Scientifique	35
B Poster Scientifique	38

Table des figures

1.1	Représentation schématique du principe de signal vibratoire.	8
1.2	Comparaison architecturale : Cloud Computing traditionnel (à gauche) vs Edge Computing moderne (à droite), décentralisant le traitement des données. Source : Digi International.	8
1.3	Cycle de vie type du TinyML, illustrant les phases distinctes d'entraînement, de conversion/optimisation, et d'inférence locale. Source : Harsha Yelchuri / Medium.	9
2.1	Schéma éclaté d'un roulement à billes standard mettant en évidence ses composants mécaniques principaux soumis à l'usure.	12
2.2	Schéma de principe illustrant la structure globale en sablier d'un réseau Autoencodeur, avec sa phase de compression et de reconstruction. Source : MathWorks.	14
3.1	Représentation schématique détaillée du flux d'inférence séquentiel et des blocs logiques décisionnels de l'architecture combinatoire.	18
3.2	Photographie de la carte de développement ESP32 sur laquelle les différents modèles ont été physiquement implémentés et profilés.	19
3.3	Organigramme détaillé décrivant le processus séquentiel de conversion, de quantification et d'exportation des modèles de Machine Learning en format TensorFlow Lite (TFLite) puis en tableaux de données compatibles C++.	21
4.1	Matrices de confusion du modèle supervisé Random Forest (utilisé seul) évalué sur les datasets CWRU et MaFaulDa.	25
4.2	Matrices de confusion du modèle non-supervisé Autoencodeur (utilisé seul) évalué sur les datasets CWRU et MaFaulDa.	25
4.3	Matrices de confusion de l' Architecture combinatoire proposée (AE + RF) évaluée sur les datasets CWRU et MaFaulDa.	26

Liste des tableaux

3.1	Répartition quantitative des échantillons formatés par jeu de données . . .	17
4.1	Bilan détaillé et exhaustif des 11 tests extrêmes sur les architectures étudiées (Datasets CWRU et MaFaulDa)	27
4.2	Performances Temporelles Comparatives - Dataset Standard Case Western (CWRU)	28
4.3	Performances Temporelles Comparatives - Dataset Bruit et Complexe MaFaulDa	28
4.4	Caractéristiques Profilées des Modèles Embarqués	29

Chapitre 1

Introduction

1.1 Contexte et Enjeux de l'Industrie 4.0

L'Industrie 4.0 marque une transition technologique et organisationnelle majeure vers des usines intelligentes, automatisées et hyper-connectées. Dans ce contexte de numérisation massive, la disponibilité et la fiabilité des équipements de production, et plus particulièrement des machines tournantes telles que les moteurs électriques, les pompes hydrauliques et les compresseurs, constituent une priorité absolue pour le maintien de la compétitivité. Un arrêt de production inattendu engendre des coûts financiers directs et indirects souvent critiques pour les industriels, se chiffrant rapidement en milliers d'euros par heure, tant en termes de réparations d'urgence à effectuer qu'en pertes d'exploitation liées à l'interruption de la chaîne de valeur logistique.

Historiquement, la maintenance industrielle reposait sur deux piliers principaux : les interventions réactives, qui consistent à réparer la machine une fois la panne matériellement survenue, ou les interventions préventives systématiques, effectuées à des intervalles réguliers fixés par le constructeur, mais qui conduisent souvent au remplacement coûteux de pièces encore parfaitement fonctionnelles. Aujourd'hui, grâce à la prolifération des capteurs à bas coût et à l'augmentation des capacités de traitement de données, l'objectif est de transiter massivement vers une **maintenance prédictive**. Cette stratégie proactive repose sur la surveillance continue et en temps réel de l'état de santé physique des équipements afin d'anticiper la dégradation des composants et de planifier les interventions chirurgicales juste avant l'occurrence d'une panne critique, optimisant ainsi considérablement la durée de vie globale des machines et minimisant les temps d'arrêt de la production.

1.2 Concepts Clés : Analyse Vibratoire, Edge AI et TinyML

Pour mettre en œuvre efficacement cette maintenance prédictive au sein d'environnements industriels complexes, notre approche technologique s'articule autour de plusieurs concepts scientifiques et matériels fondamentaux :

- **L'Analyse Vibratoire** : L'altération de la signature vibratoire d'un équipement mécanique est généralement le tout premier symptôme physique et mesurable d'une dégradation matérielle en cours, survenant de manière précoce, bien avant l'apparition de variations de température anormales ou de bruits audibles par l'opérateur humain. L'analyse de ces micro-vibrations captées par des accéléromètres à haute fréquence, couplée à des algorithmes de traitement du signal dédiés, permet d'établir un diagnostic extrêmement précoce et d'isoler l'origine mécanique de la faille.

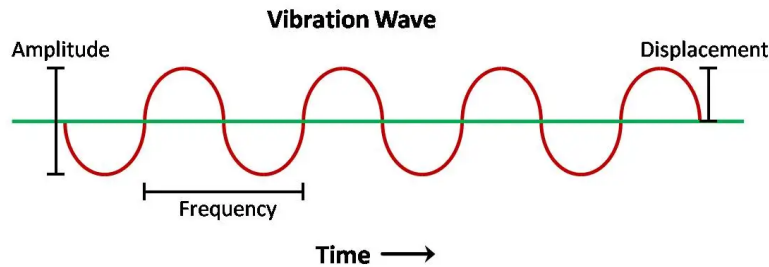


FIGURE 1.1 – Représentation schématique du principe de signal vibratoire.

- **L'Edge Computing** : Transmettre en continu des flux massifs de données vibratoires échantillonnées à plusieurs dizaines de kilohertz vers des serveurs Cloud délocalisés pose des défis architecturaux majeurs. Cela implique une latence réseau incompatible avec les arrêts d'urgence, une forte dépendance à la stabilité de la connexion internet de l'usine, des coûts de bande passante prohibitifs pour des flottes entières de machines, et des risques de cybersécurité non négligeables liés à l'exfiltration de données industrielles hautement sensibles. L'Edge Computing résout intrinsèquement ce problème en déplaçant l'intelligence et la puissance de calcul analytique "à la périphérie" directe du réseau, au plus près physique des capteurs générant la donnée.

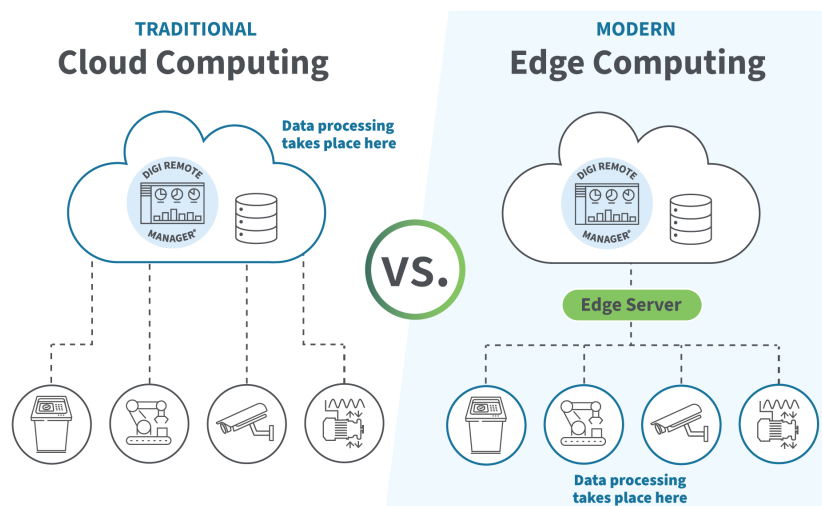


FIGURE 1.2 – Comparaison architecturale : Cloud Computing traditionnel (à gauche) vs Edge Computing moderne (à droite), décentralisant le traitement des données. Source : Digi International.

- **Le TinyML (Tiny Machine Learning)** Il s'agit d'une branche spécifique et émergente de l'Intelligence Artificielle qui consiste à concevoir, compresser, optimiser et exécuter des modèles d'apprentissage automatique complexes directement sur des systèmes embarqués disposant de très faibles ressources matérielles (quelques centaines de kilo-octets de RAM) et énergétiques, tels que de simples microcontrôleurs opérant à la périphérie (Edge).

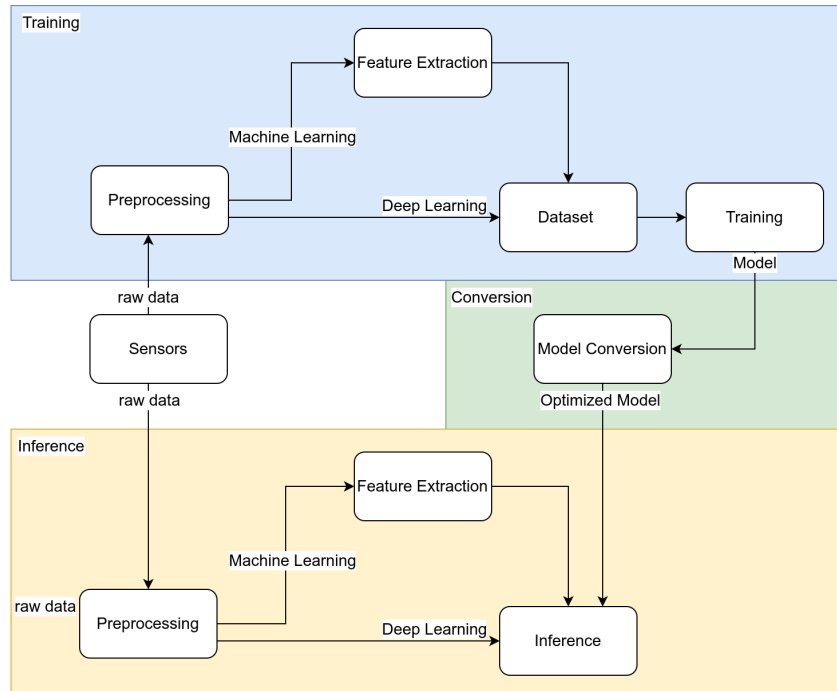


FIGURE 1.3 – Cycle de vie type du TinyML, illustrant les phases distinctes d'entraînement, de conversion/optimisation, et d'inférence locale. Source : Harsha Yelchuri / Medium.

1.3 Cadre du Stage et Collaboration de Recherche

Ce stage de fin d'études s'inscrit dans le cadre stimulant d'une collaboration de recherche internationale établie entre le laboratoire Lab-STICC de l'Université de Bretagne Occidentale (UBO) en France et l'Université de São Paulo (USP) de São Carlos au Brésil, se déroulant en lien étroit et en synergie avec les travaux de doctorat menés par Ana Fernanda Quaresma.

Ses recherches doctorales visent à concevoir et à développer des modèles d'apprentissage automatique globaux, fiables et optimisés spécifiquement pour répondre aux problématiques de la maintenance prédictive, en partenariat direct avec l'entreprise brésilienne Tractian. Les données massives issues de ce partenariat industriel serviront à terme au perfectionnement des algorithmes de surveillance déployés dans leurs propres usines clientes. Bien que des restrictions légales strictes et légitimes liées à la politique de confidentialité des données industrielles au Brésil n'aient pas permis l'exploitation directe du jeu de données propriétaire de Tractian dans le strict cadre de ce stage, la supervision, le partage de connaissances théoriques et l'aide précieuse d'Ana lors de ma mobilité à São Carlos ont été déterminantes pour la bonne orientation de mes travaux.

Cette situation de contrainte d'accès aux données a finalement permis d'orienter nos travaux respectifs de manière stratégique et complémentaire. Ainsi, tandis que les recherches d'Ana se concentrent sur l'élaboration de modèles industriels globaux et massivement entraînés pour l'écosystème Cloud de Tractian, l'angle d'attaque choisi pour ce projet de stage s'est spécifiquement et profondément ancré dans l'Edge AI et l'intégration TinyML. L'objectif commun reste invariablement la résolution des mêmes problématiques de détection et de diagnostic précoce de pannes, mais appliquées ici à des systèmes cyber-physiques critiques imposant des contraintes de temps réel strictes, le tout validé et benchmarké de manière rigoureuse sur des bases de données publiques de référence internationale.

1.4 Objectifs du Stage et Plan du Rapport

Fort de ce riche contexte collaboratif et des défis inhérents à l'intégration de l'intelligence artificielle sur des cibles matérielles limitées, ce stage a pour objectif fondamental de concevoir, d'entraîner, d'optimiser mathématiquement et d'embarquer une architecture logicielle d'IA robuste et autonome pour le diagnostic de pannes directement sur un microcontrôleur commercial de type ESP32.

Le présent rapport, qui synthétise l'ensemble des recherches et des développements effectués, s'articule comme suit :

- Le **Chapitre 2** détaille de manière approfondie le contexte scientifique sous-jacent à l'analyse vibratoire des roulements, ainsi que la théorie mathématique et le fonctionnement conceptuel des modèles d'IA (supervisés et non-supervisés) sélectionnés pour cette étude comparative.
- Le **Chapitre 3** expose la méthodologie complète de traitement des signaux temporels, les stratégies d'optimisation et de compression structurelle retenues pour permettre le déploiement sur une cible fortement contrainte comme l'ESP32, et détaille le fonctionnement architectural du code source développé durant le stage.
- Le **Chapitre 4** présente l'évaluation expérimentale exhaustive de nos modèles face à divers scénarios de défaillances, l'analyse comparative détaillée de leurs performances (accuracy, temps d'inférence, occupation mémoire), et culmine par la présentation et la validation de l'architecture algorithmique proposée finale en deux étages.
- Enfin, le **Chapitre 5** conclut ce document en dressant le bilan critique des résultats obtenus vis-à-vis des objectifs initiaux et ouvre sur les perspectives d'évolution matérielles et logicielles à envisager pour de futurs déploiements industriels du système.

Chapitre 2

Contexte Scientifique et IA

Ce chapitre a pour vocation de présenter les fondements théoriques pluridisciplinaires nécessaires à la compréhension de notre étude. Il détaille d'une part la mécanique cinématique complexe et les origines physiques des défaillances industrielles se propageant dans les équipements, et d'autre part les principes mathématiques et structurels des différents modèles d'intelligence artificielle qui ont été sélectionnés et évalués pour assurer la détection et la classification automatisée de ces anomalies.

2.1 Mécanique des Défaillances : Le Roulement à Billes

Le composant mécanique le plus universellement utilisé, mais également le plus critique et le plus sujet aux phénomènes d'usure prématurée dans l'écrasante majorité des machines tournantes industrielles, est le roulement à billes. Il a pour rôle essentiel de supporter les charges radiales et axiales tout en réduisant drastiquement la friction entre les pièces mobiles en rotation et le châssis statique. Un roulement classique est intrinsèquement constitué de quatre parties principales distinctes, interagissant en permanence :

1. **La bague intérieure (Inner race)** : composant métallique solidaire de l'arbre moteur en rotation, subissant des contraintes cycliques intenses à chaque tour.
2. **La bague extérieure (Outer race)** : composant généralement fixe, logé rigidement dans le carter de la machine, absorbant les vibrations résiduelles.
3. **Les billes (Balls)** : les éléments roulants sphériques assurant la transmission des forces entre les deux bagues tout en minimisant la surface de contact.
4. **La cage** : la structure intercalaire qui maintient un espacement angulaire constant entre les billes, empêchant leur collision mutuelle et répartissant la charge de manière homogène.

L'apparition du moindre défaut de surface (micro-fissure due à la fatigue du métal, écaillage de la matière, usure abrasive liée à un défaut de lubrification) sur l'une de ces quatre parties génère inévitablement des micro-chocs mécaniques à chaque fois qu'un élément roulant passe sur l'anomalie lors de la rotation. Ces chocs répétés et périodiques se traduisent par l'excitation des fréquences de résonance de la structure et l'apparition de pics d'amplitude très spécifiques dans le spectre des fréquences vibratoires du moteur. Notre objectif ambitieux est non seulement de détecter l'apparition de ces défauts alors qu'ils sont encore infimes (de l'ordre de 0,18 mm, soit environ 0.007 pouce de diamètre) avant qu'ils ne se propagent et ne provoquent une rupture catastrophique, mais également de les classer avec un haut niveau de certitude selon leur origine mécanique

précise (bague interne, externe, etc.) afin d'orienter efficacement les futures opérations de maintenance.

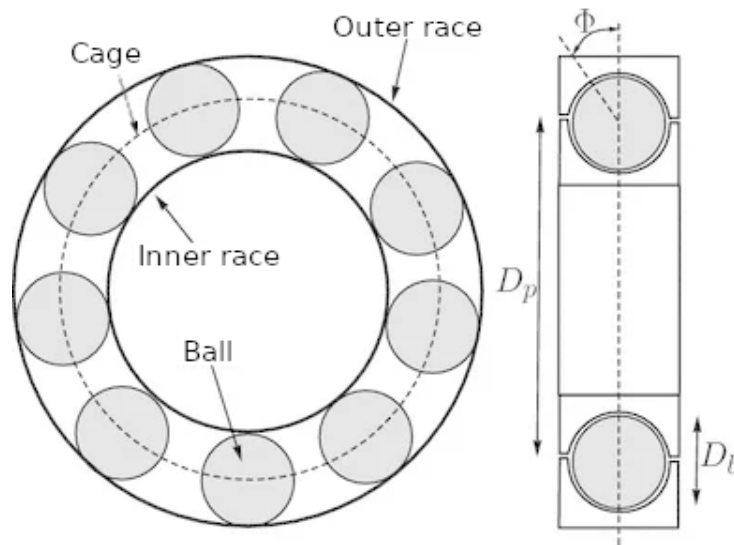


FIGURE 2.1 – Schéma éclaté d'un roulement à billes standard mettant en évidence ses composants mécaniques principaux soumis à l'usure.

2.2 Modèles d'Apprentissage Automatique Étudiés

D'un point de vue purement algorithmique, l'identification de ces signatures vibratoires altérées s'apparente à un problème complexe de classification multi-classes ou de détection d'anomalies dans des séries temporelles ou fréquentielles. Pour traiter ce problème sur une architecture embarquée, nous avons scrupuleusement sélectionné deux grandes familles de modèles présentant des caractéristiques mathématiques et opérationnelles distinctes, voire opposées.

2.2.1 L'approche Supervisée : Random Forest (RF)

Le Random Forest (ou Forêt Aléatoire) est un algorithme d'apprentissage supervisé particulièrement robuste basé sur la technique d'ensemblisme (bagging), agrégeant les prédictions d'une multitude d'arbres de décision construits de manière pseudo-aléatoire. Lors de sa phase d'entraînement, il apprend à partitionner et à séparer l'espace vectoriel des données en se basant sur un vaste corpus d'exemples préalablement et minutieusement étiquetés par des experts pour chaque type de panne mécanique connue. Dans notre implémentation spécifique visant le compromis entre performance et légèreté, le Random Forest est configuré avec 80 arbres de décision (estimators), limités à une profondeur maximale de 10 nœuds conditionnels, et intégrant une pondération des classes équilibrée pour éviter les biais majoritaires.

- **Avantages :** Ce modèle est computationnellement extrêmement léger lors de l'inférence. Il offre une excellente précision statistique (accuracy) sur les classes de pannes qu'il a rencontrées lors de l'entraînement, et sa structure fondamentalement basée sur des tests logiques conditionnels de type (if/else) s'avère parfaitement adaptée à l'architecture intrinsèque d'un processeur de microcontrôleur.

- **Inconvénients** : Sa principale faiblesse réside dans le fait qu'il requiert impérativement de posséder des jeux de données massifs et annotés pour toutes les pannes possibles et imaginables, ce qui est une condition souvent totalement irréaliste dans la réalité des milieux industriels où les pannes sont par nature rares. De plus, lorsqu'il est inévitablement confronté à un signal défectueux totalement inédit (par exemple, la casse simultanée de deux composants), le Random Forest réalisera ce que l'on appelle une "classification forcée" en l'assignant arbitrairement à la classe connue qu'il juge géométriquement la plus proche, générant ainsi un diagnostic formel mais factuellement faux et potentiellement dangereux.

2.2.2 L'approche Non-Supervisée : Autoencodeurs (AE et VAE)

Afin de pallier ce problème critique de la classification des pannes inconnues et de garantir une sécurité de détection absolue, nous avons exploré le domaine des réseaux de neurones profonds non-supervisés, et plus spécifiquement la topologie des Autoencodeurs. Contrairement au RF, ces modèles sont entraînés uniquement sur des bases de données massives représentatives du fonctionnement parfaitement sain et nominal de la machine. Ils apprennent ainsi de manière autonome à modéliser la "normalité" vibratoire et à en extraire les caractéristiques intrinsèques sans aucune intervention humaine d'étiquetage.

- **Autoencodeur standard (AE) [1]** : Sa structure le force à compresser les données d'entrée complexes dans un espace mathématique latent de dimension fortement réduite (via la partie encodeur), puis il tente de reconstruire le signal d'origine le plus fidèlement possible à partir de cette représentation compressée (via la partie décodeur). L'erreur de reconstruction finale, mesurée le plus souvent par l'Erreur Quadratique Moyenne (MSE), sert d'indicateur direct de la santé du système. Si le signal entrant contient une anomalie mécanique (une vibration jamais vue lors de l'apprentissage), le modèle échouera logiquement à le faire passer à travers son goulot d'étranglement et à le reconstruire correctement, faisant mécaniquement exploser la valeur de la MSE bien au-delà d'un seuil d'alerte prédéfini.
- **Autoencodeur Variationnel (VAE) [3]** : Cette architecture constitue une variante probabiliste et générative de l'AE standard. Le VAE cartographie les données d'entrées non pas vers des vecteurs de points fixes et déterministes, mais selon des distributions statistiques complexes définies par une moyenne et une variance. Cette contrainte de régularisation forte permet de structurer un espace latent mathématiquement plus continu et lisse, améliorant théoriquement la robustesse du modèle face aux bruits transitoires et à l'overfitting, au prix d'une complexité mathématique de calcul (notamment l'échantillonnage de la divergence de Kullback-Leibler) et d'une empreinte matérielle bien supérieures.

Initialement au cours de nos recherches, l'Autoencodeur Variationnel (VAE) avait été fortement envisagé comme la solution principale pour agir en tant que diagnostiqueur global. L'hypothèse de départ stipulait que son espace latent probabiliste serait particulièrement efficace pour éviter les erreurs de classification face à des signatures vibratoires de pannes totalement inconnues, un rôle aujourd'hui dévolu au Random Forest. Néanmoins, les évaluations expérimentales n'ont pas été concluantes et ont même démontré des performances de reconstruction et de discrimination parfois inférieures à celles de l'Autoencodeur standard sur notre problématique spécifique. De plus, la complexité mathématique inhérente à l'échantillonnage de la distribution latente rend le VAE excessivement lourd en termes de calculs pour l'ESP32, en particulier lorsqu'il s'agit de différencier

plusieurs classes de défauts simultanément. Par conséquent, cette piste probabiliste a été définitivement écartée.

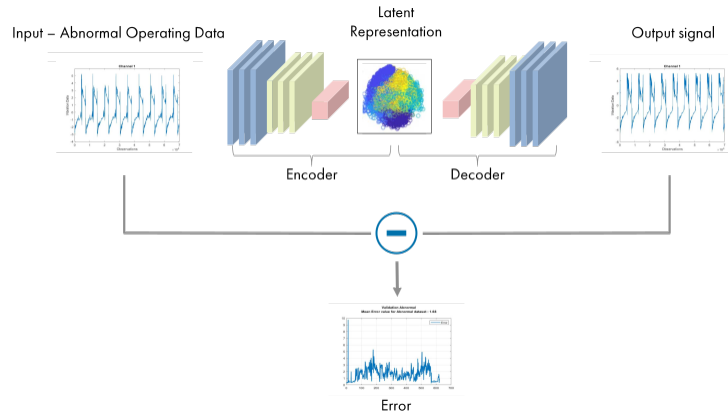


FIGURE 2.2 – Schéma de principe illustrant la structure globale en sablier d'un réseau Autoencodeur, avec sa phase de compression et de reconstruction. Source : MathWorks.

Concrètement, l'intégration de telles architectures d'apprentissage profond soulève des interrogations matérielles et logicielles décisives. La première problématique a consisté à évaluer la faisabilité du déploiement d'un Autoencodeur directement sur la mémoire très restreinte de l'ESP32. La seconde interrogation, corollaire de la première, a porté sur la possibilité d'embarquer ce modèle de manière "brute" (c'est-à-dire en conservant l'intégralité de sa précision originelle en virgule flottante) ou sur la nécessité absolue d'appliquer des techniques de compression, telles que la quantification des poids synaptiques ou le pruning. Nos recherches ont formellement prouvé que le recours à de telles optimisations n'est pas nécessaire pour espérer s'exécuter dans le milieu hyper-contraint de l'informatique embarquée. S'il demeure incontestable que des choix de conception structurelle (comme la limitation du goulot d'étranglement) et un prétraitement des signaux en amont sont nécessaires pour limiter l'empreinte initiale, l'exécution opérationnelle du modèle brut s'est avérée possible, offrant une fluidité et une fiabilité remarquables sans avoir eu besoin de sacrifier la justesse mathématique du réseau.

Notre Autoencodeur optimisé intègre ainsi des couches d'extraction de caractéristiques basées sur la convolution 1D (Conv1D) pour capturer les motifs locaux du signal temporel, couplées à une couche dense centrale formant un goulot d'étranglement sévère, réduit à seulement 16 dimensions continues. La phase cruciale d'entraînement intègre une fonction d'arrêt précoce (Early Stopping) monitorant l'erreur de validation avec une patience fixée à 8 époques afin d'empêcher efficacement tout phénomène de surapprentissage des données saines.

De surcroît, une approche par algorithme de clustering non supervisé de type K-Means (testant itérativement des configurations allant de 2 à 7 clusters) a été initialement intégrée pour intervenir spécifiquement sur cet espace latent de 16 dimensions. L'objectif visé était de segmenter les différents régimes de profils sains sous-jacents, permettant d'y associer des seuils d'erreur MSE adaptatifs beaucoup plus fins et précis qu'un seuil d'alerte global unique. Il convient toutefois de préciser que l'activation de ce module K-Means a été rendue optionnelle et conditionnelle au sein du code source. Les analyses comparatives approfondies des performances (avec et sans cette étape de clustering) ont finalement révélé que le gain théorique d'accuracy apporté ne compensait pas la charge calculatoire

ajoutée. Pour le rendu final de l'architecture, l'utilisation des K-Means n'a donc pas été retenue comme satisfaisante, l'approche par seuil global s'étant avérée tout aussi robuste.

Afin de réduire drastiquement la complexité d'entrée du réseau, les signaux temporels bruts subissent systématiquement une Transformée de Fourier Rapide (FFT) en amont pour extraire et lisser les caractéristiques énergétiques utiles du spectre. Par la suite, le code complet repose sur des techniques d'optimisation logicielles poussées, telles que la quantification des poids pour les réseaux de neurones (réduction de précision matérielle) ou la transpilation native et arborescente en langage C++ pour les modèles d'arbres de décision.

Chapitre 3

Méthodologie et Déploiement Embarqué (TinyML)

3.1 Acquisition et Prétraitement des Données

Pour entraîner rigoureusement et évaluer objectivement la capacité de généralisation de nos différents modèles d’Intelligence Artificielle, nous nous sommes appuyés sur la fusion de plusieurs bases de données de référence internationalement reconnues dans le domaine du Pronostic and Health Management (PHM) :

- **Dataset CWRU (Case Western Reserve University) [2]** : Il s’agit du standard académique de facto, contenant des données acquises en laboratoire dans des conditions très contrôlées où les défauts mécaniques sont créés artificiellement par des techniques précises d’électroérosion (induisant des trous calibrés de 0,18 mm / 0.007 pouce). Les signatures vibratoires générées y sont très distinctes et présentent un rapport signal-sur-bruit élevé, ce qui les rend idéales pour valider les concepts algorithmiques de base. Ce dataset permet l’étude des signaux parfaitement sains, ainsi que des défauts isolés localisés sur la cage, sur les billes ou sur la bague externe du roulement.
- **Dataset MaFaulDa (Machinery Fault Database) [4]** : Issu d’un banc de test et simulateur mécanique universitaire beaucoup plus complexe intégrant des déséquilibres de charge et des défauts d’alignement, ce dataset présente des usures souvent entrelacées et des signatures vibratoires extrêmement subtiles et noyées dans le bruit de fonctionnement, rendant l’isolation spectrale de la panne beaucoup plus difficile et constituant un excellent test de robustesse.
- **Données Tractian (Non exploitées)** : Initialement, le cahier des charges prévoyait l’exploitation de données de télémétrie issues de l’industrie réelle, captées directement sur des équipements en production par l’entreprise partenaire Tractian. L’objectif de cette démarche était de valider de manière purement empirique la transférabilité et l’applicabilité pratique de nos approches algorithmiques en dehors du cadre idéalisé et maîtrisé du laboratoire. Néanmoins, en raison de clauses de confidentialité extrêmement strictes et de barrières liées à la législation brésilienne encadrant la protection de la propriété intellectuelle des données industrielles, l’accès direct et l’utilisation de ce jeu de données propriétaire se sont finalement révélés impossibles dans le cadre temporel et légal imparti à ce stage.

Le signal vibratoire temporel brut capté par l'accéléromètre, qui est une onde oscillante complexe comprenant des milliers de points par seconde, est très difficilement exploitable directement par de petits réseaux de neurones en raison de sa dimensionnalité écrasante. Les signaux temporels en continu sont donc rigoureusement fenêtrés en blocs d'analyse discrets de 512 échantillons, sur un intervalle de chevauchement de 100 échantillons pour garantir la continuité temporelle de l'analyse. Ces blocs de données temporelles sont ensuite mathématiquement transformés dans le domaine fréquentiel via l'application d'une Transformée de Fourier Rapide (FFT). La composante continue du signal résultant, ou bin 0, qui représente l'offset du capteur plutôt qu'une vibration réelle, est systématiquement retirée en raison de son apport informatif quasi-nul pour le diagnostic. Ensuite, une transformation logarithmique non linéaire est appliquée sur l'ensemble du spectre afin de rééquilibrer les échelles, ce qui a pour effet d'accentuer la visibilité des harmoniques mineures cruciales tout en compressant l'écrasante amplitude des grands pics de rotation fondamentale. Enfin, une normalisation globale de type Min-Max est effectuée pour ramener l'ensemble des valeurs d'entrée du réseau dans un espace vectoriel strictement compris entre 0 et 1, facilitant ainsi la convergence des gradients lors de l'apprentissage.

Afin d'éprouver la robustesse intrinsèque de l'extraction de caractéristiques opérée par nos modèles face aux aléas de l'environnement industriel, une technique de *Data Augmentation* sévère par injection de bruit gaussien aléatoire est appliquée de manière dynamique lors de la phase d'entraînement. Concernant la structuration de notre corpus d'étude (dont la volumétrie exacte est détaillée dans le Tableau 3.1), la séparation des données a été effectuée selon un ratio précis : 60 % des échantillons temporels extraits sont dédiés à la phase stricte d'entraînement des poids du modèle ($\mathbf{x}_{\text{train}}$), 20 % sont isolés de manière stratifiée pour former un jeu de validation ($\mathbf{x}_{\text{val}}$) servant spécifiquement à la calibration intelligente des seuils MSE de l'Autoencodeur, et enfin, les 20 % restants sont rigoureusement sanctuarisés dans un jeu de test indépendant ($\mathbf{x}_{\text{test}}$) pour l'évaluation finale et impartiale des performances.

TABLE 3.1 – Répartition quantitative des échantillons formatés par jeu de données

Jeu de données	Total	Entraînement (60%)	Validation (20%)	Test (20%)
Case Western (CWRU)	4 874	2 924	975	975
MaFaulDa	2 494	1 496	499	499

** Note méthodologique : Le jeu d'entraînement ($\mathbf{x}_{\text{train}}$) permet l'apprentissage des poids synaptiques ou des arbres de décision. Le jeu de validation ($\mathbf{x}_{\text{val}}$) est exploité pour la calibration algorithmique du seuil de détection d'anomalie. Le jeu de test ($\mathbf{x}_{\text{test}}$) est exclusivement réservé à l'évaluation terminale des métriques.*

3.2 Architecture Combinatoire Proposée

Face au constat incontestable de ce compromis technique persistant (Précision sur données connues contre capacité de généralisation sécuritaire sur l'inconnu), notre contribution scientifique principale réside dans la conception et le développement de l'architecture combinatoire alliant Autoencodeur et Random Forest. Cette solution algorithmique fonctionne en deux étages distincts et complémentaires, conçus pour s'exécuter séquentiellement de manière asynchrone sur l'ESP32 afin d'essayer de tirer le meilleur des deux modèles :

1. **Étage 1 (Filtre de Normalité Universel)** : Chaque nouveau bloc de signal FFT capté passe systématiquement et prioritairement dans le réseau de l'Autoencodeur. La valeur scalaire de la MSE de reconstruction est alors comparée à un seuil d'erreur. Contrairement à une approche statique, le choix de ce seuil fait l'objet d'une calibration intelligente lors de l'apprentissage : le système évalue un seuil global unique et le compare à une stratégie de seuils adaptatifs basés sur un clustering K-Means de l'espace latent. L'approche par clusters n'est conservée que si elle permet d'améliorer le F1-Score d'au moins une marge minimale (définie à 0,01 dans notre script), faute de quoi le seuil global est privilégié pour sa simplicité et son efficacité. Si la MSE de l'échantillon dépasse le seuil finalement retenu, le signal est alors flaggué comme "Anomalie". Dans ce rôle critique, l'Autoencodeur agit comme un gardien universel pour détecter les comportements déviants, et en particulier ceux de nature inconnue. S'il n'y a pas dépassement de ce seuil, le processus s'arrête immédiatement et le système est déclaré sain, ce qui permet d'économiser de précieuses ressources de calcul.
2. **Étage 2 (Diagnostic Expert et Spécifique)** : Les données du signal FFT d'entrée sont alors transmises et injectées dans les nœuds du Random Forest pour un examen approfondi. Ce dernier va tenter, à la lumière de ses exemples appris, de classifier formellement la panne sous-jacente afin de fournir un diagnostic de maintenance précis (ex : bague interne, bague externe). Fait crucial de notre implémentation, le Random Forest possède également la capacité de contredire l'Autoencodeur : si l'arbre de décision estime que l'échantillon appartient à la classe nominale 0 (Sain), il corrige le doute initial de l'Autoencodeur et annule l'alerte, requalifiant définitivement le résultat comme normal. Enfin, concernant la prise de décision sur les pannes, le code Python implémenté s'appuie directement sur la fonction mathématique *argmax* pour extraire la classe possédant la probabilité d'appartenance la plus élevée. Ainsi, notre script Python effectue une classification forcée en retenant systématiquement la classe la plus probable, même si la forêt d'arbres est confuse face à une défaillance inconnue. Le taux de confiance final est néanmoins extrait des probabilités et affiché pour informer l'opérateur de la fiabilité de la classification lors du diagnostic.

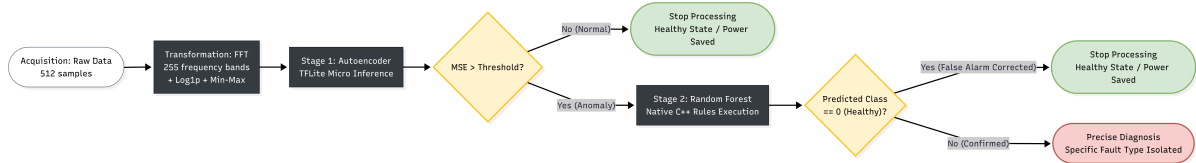


FIGURE 3.1 – Représentation schématique détaillée du flux d'inférence séquentiel et des blocs logiques décisionnels de l'architecture combinatoire.

3.3 Contraintes Matérielles : La cible ESP32

L'étape finale de déploiement de ces architectures algorithmiques s'effectue directement sur un microcontrôleur ESP32, un SoC (System on a Chip) devenu un standard

absolu de l’IoT industriel en raison de son rapport coût-performance et de sa connectivité embarquée. Cependant, ses spécifications techniques imposent des limites physiques drastiques pour quiconque souhaite y faire tourner de l’Intelligence Artificielle :

- **SRAM (Static Random Access Memory)** : Il dispose d’environ 520 Ko au total, partagés entre le système d’exploitation temps réel (RTOS), les piles réseau Wi-Fi/Bluetooth, les variables dynamiques de l’application et le tas (heap). Cette limite est le goulot d’étranglement majeur empêchant le chargement de gros tenseurs de données en mémoire vive.
- **Mémoire Flash** : Limitée à 4 Mo, elle est dédiée au stockage pérenne du firmware compilé, du code du programme et des poids quantifiés des réseaux de neurones, obligeant les modèles à conserver une empreinte de stockage minimale.
- **CPU** : Architecturé autour d’un processeur Xtensa dual-core cadencé à une fréquence maximale de 240 MHz, sans accélérateur matériel d’intelligence artificielle dédié (NPU), imposant que les calculs matriciels soient effectués de manière logicielle et séquentielle par le processeur principal.

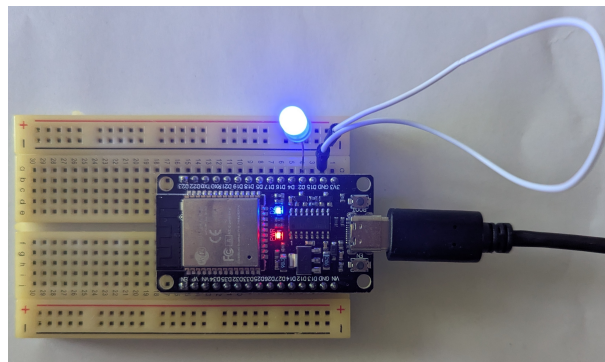


FIGURE 3.2 – Photographie de la carte de développement ESP32 sur laquelle les différents modèles ont été physiquement implémentés et profilés.

3.4 Optimisation et Transpilation (TinyML)

Les modèles d’apprentissage profond traditionnellement développés et entraînés sur des stations de travail avec des frameworks comme Keras ou PyTorch en Python manipulent des poids en virgule flottante très gourmands et sont beaucoup trop volumineux pour être intégrés tels quels sur la mémoire limitée de l’ESP32. Des processus de conversion rigoureux et spécifiques au paradigme du TinyML ont donc été mis en place pour pallier ces barrières matérielles.

3.4.1 Compression de l’Autoencodeur (TFLite Micro)

L’Autoencodeur est d’abord drastiquement compressé via l’utilisation de l’API de conversion TensorFlow Lite Converter. Cette étape cruciale se charge d’aplatir (flatten) le graphe de calcul complexe du réseau de neurones Keras/TensorFlow en un format binaire statique et ultra-compact, spécialement conçu pour les architectures sans système d’exploitation. Dans le cadre de cette implémentation initiale, et comme l’illustre l’organigramme de notre flux de travail (Figure 3.3), le modèle converti conserve sa précision

d'origine en virgule flottante 32 bits (Float32). Ce choix d'implémentation a été délibérément arrêté dans le but scientifique de démontrer formellement qu'il est tout à fait possible d'embarquer et d'exécuter un réseau de neurones profond de type Autoencodeur directement sur un microcontrôleur aussi contraint que l'ESP32, et ce, sans même avoir besoin de recourir d'emblée à des optimisations de compression drastiques. En prouvant empiriquement que l'architecture matérielle est capable de supporter la charge computationnelle et mémorielle d'un modèle brut et non altéré, nous validons la viabilité fondamentale de l'Edge AI pour cette tâche spécifique, prouvant que le goulot d'étranglement matériel n'est pas aussi infranchissable qu'il n'y paraît.

Pour éviter totalement d'avoir recours à une carte mémoire SD externe pour stocker le modèle – ce qui serait beaucoup trop lent pour une lecture en temps réel et très sujet aux déconnexions mécaniques dues aux vibrations intenses de la machine rotative –, le modèle compressé au format binaire '.tflite' est directement sérialisé sous la forme d'un grand tableau statique constant écrit en langage C (processus de *Hex Dump to C-Array*). De la même manière, les vecteurs contenant les valeurs de seuils MSE calculés pour chaque cluster sont exportés de manière totalement automatique vers des tableaux d'en-tête C pour une inclusion directe lors de la compilation matérielle.

Il est toutefois essentiel de souligner que des marges d'optimisation logicielles drastiques restent volontairement inexploitées à ce stade et constituent des perspectives d'évolution majeures ("Future Considerations"). En effet, l'application d'une étape de **Quantization** (quantification post-entraînement visant à convertir les poids Float32 en entiers Int8) ainsi que le **Pruning** (l'élagage mathématique des connexions neuronales redondantes) n'ont pas encore été appliquées dans le script de génération actuel. Ces opérations futures permettraient de diviser mathématiquement la taille globale du modèle stocké par 4 et d'accélérer encore davantage les opérations matricielles du processeur embarqué, au prix d'une perte d'accuracy infime. Du côté du Random Forest, une optimisation similaire par *Cost-Complexity Pruning* (élagage des branches de décision superflues) est également identifiée pour minimiser la taille finale du code C++ généré. L'architecture actuelle démontre donc avec succès la faisabilité conceptuelle du système Edge AI, tout en laissant une marge de manœuvre considérable pour des réductions d'empreinte mémoire ultérieures en vue d'une industrialisation.

Une fois dans l'environnement de développement ciblant le microcontrôleur comme l'IDE Arduino (utilisant les langages C/C++), l'inférence est gérée par la bibliothèque spécialisée *TensorFlow Lite for Microcontrollers*. Le système est conçu de telle sorte qu'il alloue statiquement au démarrage un bloc de mémoire vive pré-défini et dédié exclusivement aux calculs intermédiaires des tenseurs, un espace appelé **Tensor Arena**. Cette gestion déterministe de la mémoire garantit qu'aucune allocation dynamique dangereuse n'a lieu en cours d'exécution, assurant mathématiquement qu'aucun dépassement de pile ou de tas (Stack/Heap Overflow) ne viendra crasher le microcontrôleur durant la phase critique d'inférence continue.

3.4.2 Transpilation native du Random Forest

Pour le modèle de Random Forest, une approche d'optimisation encore plus radicale et innovante a été retenue par rapport à l'utilisation d'un framework d'IA classique. Plutôt que d'embarquer un moteur d'inférence lourd et complexe pour lire la structure de la forêt, le modèle préalablement entraîné sous Python (avec ses hyper-paramètres d'élagage fixés pour limiter drastiquement la profondeur maximale de l'arbre et le nombre global de

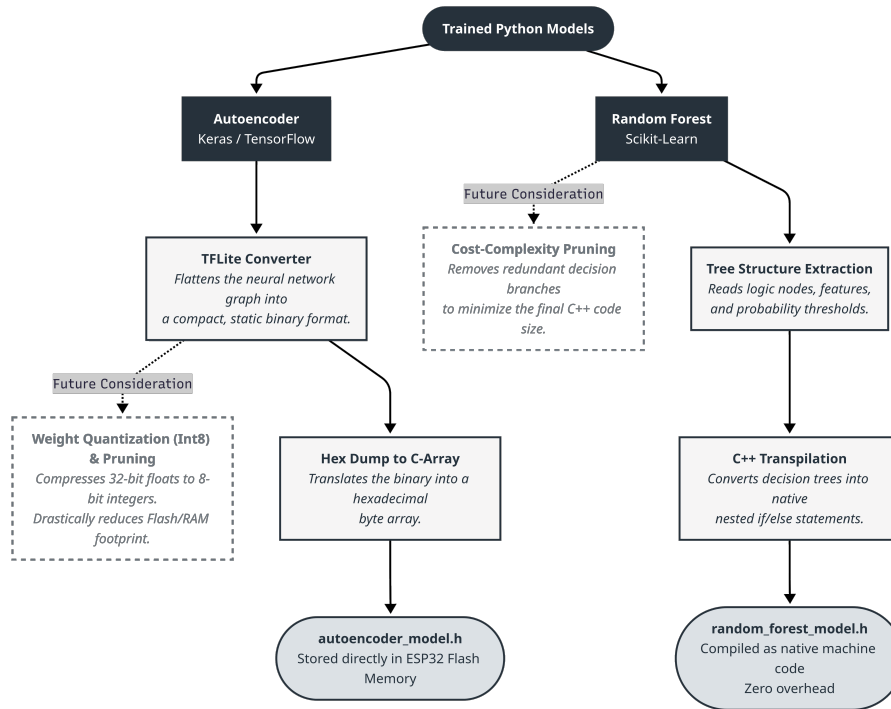


FIGURE 3.3 – Organigramme détaillé décrivant le processus séquentiel de conversion, de quantification et d’exportation des modèles de Machine Learning en format TensorFlow Lite (TFLite) puis en tableaux de données compatibles C++.

branches évaluées) est directement et intégralement transpilé (converti source à source) en une gigantesque suite de règles conditionnelles de type ‘if/else’ profondément imbriquées, générant un fichier de code source en langage C++ pur et natif. Cette fonction C++ colossale va virtuellement parcourir les chemins de décision et synthétiser la moyenne exacte des vecteurs de probabilités qui auraient été émis par l’exécution des 80 arbres du modèle original. Cette méthode d’ingénierie logicielle affranchit purement et simplement le microcontrôleur de la nécessité de charger en RAM toute bibliothèque externe d’intelligence artificielle, offrant ainsi une empreinte mémoire vive virtuellement quasi-nulle et un temps d’exécution qui bat tous les records, car le code résultant bénéficie directement des optimisations du compilateur GCC natif de l’ESP32.

3.5 Défis Techniques et Difficultés Rencontrées

Le développement et l’implémentation de cette architecture Edge AI ont été jalonnés par plusieurs obstacles majeurs, inhérents à la transition entre la théorie de l’apprentissage automatique et la réalité physique des systèmes embarqués.

Premièrement, l’accès aux données industrielles a constitué un point de blocage organisationnel inattendu. Les clauses de confidentialité strictes et la législation brésilienne sur la propriété intellectuelle ont finalement empêché l’exploitation du jeu de données propriétaire de l’entreprise Tractian. Cette restriction stricte a imposé une implémentation concentrée sur les bases de données publiques (CWRU et MaFaulDa).

Deuxièmement, les contraintes mémorielles de l’ESP32 ont posé un défi logiciel constant. L’allocation statique de la *Tensor Arena* exigée par TensorFlow Lite Micro pour l’Autoencodeur a nécessité de multiples itérations empiriques. L’objectif était de trouver la taille

mémoire exacte permettant l'inférence des couches denses sans provoquer de dépassement de capacité fatidique (Stack Overflow), tout en laissant impérativement suffisamment de SRAM disponible pour le fonctionnement du système d'exploitation temps réel (RTOS) et l'exécution de la volumineuse fonction C++ du Random Forest transpilé.

Enfin, sur le plan purement algorithmique, la calibration croisée de l'architecture combinatoire a été particulièrement fastidieuse. Trouver le point d'équilibre mathématique exact où le Random Forest corrige efficacement les fausses alarmes de l'Autoencodeur, sans pour autant écraser sa capacité vitale à détecter des micro-défauts, a exigé des dizaines d'heures de simulation et d'ajustements des hyperparamètres avant d'arriver au constat analytique formel présenté dans le chapitre des résultats.

3.6 Disponibilité et Utilisation du Code Source

Dans une forte volonté de s'inscrire dans une démarche de science ouverte, de garantir la reproductibilité totale de ces travaux de recherche par la communauté scientifique, et d'en faciliter la prise en main par d'éventuels partenaires industriels, l'intégralité du pipeline de traitement des données, d'entraînement et d'exportation a été minutieusement documentée, nettoyée et publiée sur un dépôt GitHub public, accessible librement à l'adresse suivante : https://github.com/Alexandre1422/BearingFaultDetection_Stage_M2.

L'architecture logicielle du projet s'articule élégamment autour d'un script Python unique, centralisé et modulaire, judicieusement nommé `implementation_complete.py`. Ce script orchestre et exécute séquentiellement l'ensemble du flux de travail du Data Scientist : allant du chargement dynamique et du nettoyage des jeux de données bruts (qu'il s'agisse de fichiers '.mat' de Matlab ou de fichiers tabulaires '.csv'), en passant par les lourdes étapes d'extraction des caractéristiques fréquentielles (application de la FFT, compression logarithmique, normalisation min-max), jusqu'à l'entraînement itératif de tous les modèles d'IA, la génération de métriques pour leur évaluation statistique, et s'achevant de manière automatisée par leur exportation finale et formatée sous la forme de fichiers d'en-tête C++ ('.h') directement prêts à être inclus pour le déploiement matériel sur l'ESP32.

Afin d'offrir une flexibilité maximale à l'utilisateur, ce script central a été conçu pour être hautement paramétrable via une interface en ligne de commande (CLI) robuste, permettant aux chercheurs de moduler les paramètres d'entraînement à la volée selon trois arguments principaux explicites :

- **- dataset** : Permet de sélectionner instantanément le jeu de données cible pour l'expérience (**cw** par défaut pour charger l'environnement Case Western, ou **mfd** pour basculer sur l'environnement plus bruité de MaFaulDa).
- **-model** : Définit l'architecture algorithmique désirée à instancier. L'option **full** (qui est l'option par défaut de notre contribution) entraîne et combine l'architecture associant l'Autoencodeur et le Random Forest complète. Les options plus spécifiques **ae** (pour Autoencodeur seul) ou **rf** (pour Random Forest seul) permettent d'isoler l'étude et d'entraîner uniquement l'un des deux modèles pour générer des benchmarks séparés.
- **-mode** : Ajuste la granularité conceptuelle de la classification de sortie souhaitée (**all** forcera le système à tenter de différencier l'état sain des trois types distincts de pannes mécaniques, tandis que l'argument **healthy** simplifiera le problème en une simple détection binaire globale de type présence/absence d'anomalie, sans catégorisation).

À l'issue de son exécution complète, qui ne prend que quelques minutes sur un ordinateur standard, le script Python se charge de générer de manière totalement automatique la panoplie complète et cohérente de fichiers de configuration nécessaires à l'inférence embarquée. On y retrouve le modèle de graphe TFLite quantifié de l'Autoencodeur (incluant logiquement la partie encodeur), les tableaux des coordonnées des centroïdes K-Means (essentiels pour le typage palliatif des pannes ou l'application des seuils adaptatifs selon les profils), les valeurs précises des seuils d'erreur MSE calibrées sur le jeu de validation, et enfin la transpilation C++ native et arborescente du Random Forest complet.

Chapitre 4

Évaluation Expérimentale et Résultats

4.1 Analyse de la Séparation Spatiale et Fréquentielle

Dans le cadre industriel exigeant de cette étude, l'objectif fondamental assigné à notre système de maintenance prédictive est résolument double : il doit être en capacité d'identifier et de classer avec certitude les pannes déjà connues et modélisées par l'entreprise, mais il doit surtout agir comme un filet de sécurité infailible pour isoler, détecter et bloquer immédiatement les pannes inconnues et inédites, évitant ainsi tout dommage collatéral non anticipé.

Afin de comprendre visuellement le comportement interne de nos modèles face à cette problématique, une Analyse en Composantes Principales (PCA) poussée a été menée pour réduire la dimensionnalité et visualiser en 2D ou 3D le vaste espace de décision cartographié par les algorithmes. Totalement dépourvu d'un mécanisme de création d'espace latent structuré par apprentissage des représentations, le modèle Random Forest est contraint d'opérer ses coupures de décision directement sur les composantes des entrées brutes. Ce mode opératoire très directif entraîne visuellement, lors des analyses de projection, un net chevauchement géométrique des différentes classes de pannes aux frontières des domaines, expliquant ses probables confusions lors de signaux légèrement bruités. À l'inverse total de cette approche, l'Autoencodeur standard projette très efficacement et de manière continue les signaux temporels complexes dans son espace goulot d'étranglement de 16 dimensions, formant des clusters géométriques de données saines qui s'avèrent être extrêmement denses, regroupés et parfaitement isolés des signaux anormaux environnants. Le modèle d'Autoencodeur Variationnel (VAE) parvient lui également à bien regrouper et structurer les différentes classes saines dans son espace latent probabiliste, mais sa nature intrinsèquement axée sur les probabilités de distribution tend à forcer un léger lissage mathématique et donc un chevauchement vaporeux des frontières entre les classes nominales, le rendant empiriquement un tout petit peu moins discriminant et incisif que l'AE standard sur la nature des anomalies de ce dataset précis.

L'étude d'importance croisée des bandes de fréquences révèle également de manière flagrante que le Random Forest prend ses décisions finales en se focalisant de manière extrêmement sélective et agressive sur l'amplitude numérique de quelques pics FFT très précis et caractéristiques (liés aux fréquences théoriques de roulements). Les différents Autoencodeurs analysent et sont forcés de reconstruire le spectre énergétique du signal dans sa quasi-globalité pour parvenir à minimiser l'erreur quadratique de reconstruction globale (MSE), ce qui leur garantit de facto une bien meilleure résilience face aux pics de bruits transitoires ou aux artefacts de mesure inévitables en usine.

4.2 Évaluation sur les Classes Nominales (Matrices de confusion)

Avant d'éprouver la robustesse absolue du système face à des scénarios de défaillances extrêmes et inattendues, il convient d'analyser en détail le comportement et l'exactitude de chaque architecture sur les données nominales de test. Ces données sont composées des états sains de la machine, des défaillances mécaniques parfaitement connues et étiquetées (Défaut de Cage, Défaut de Bille, Défaut de Bague Externe), ainsi que d'une classe piège ("Inconnu") servant de témoin.

Pour illustrer visuellement ces performances de classification, les figures suivantes présentent les matrices de confusion générées pour chacun des trois modèles étudiés. Afin de faciliter l'analyse comparative, chaque figure propose un montage réunissant côte à côte les résultats obtenus sur le dataset de laboratoire (CWRU) et sur le dataset industriel beaucoup plus bruyé (MaFaulDa).

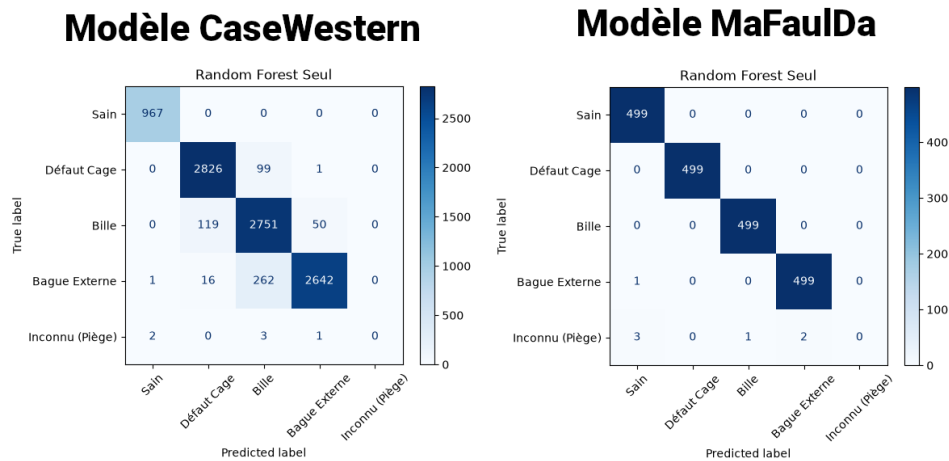


FIGURE 4.1 – Matrices de confusion du modèle supervisé **Random Forest** (utilisé seul) évalué sur les datasets CWRU et MaFaulDa.

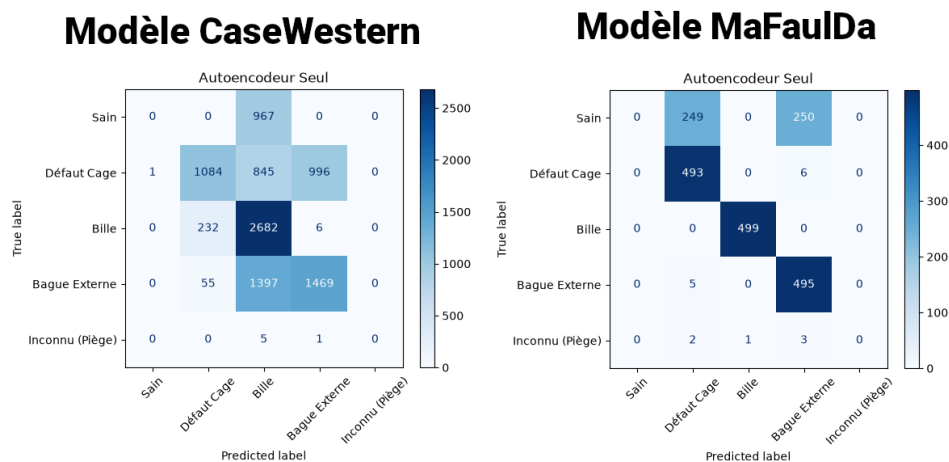


FIGURE 4.2 – Matrices de confusion du modèle non-supervisé **Autoencodeur** (utilisé seul) évalué sur les datasets CWRU et MaFaulDa.

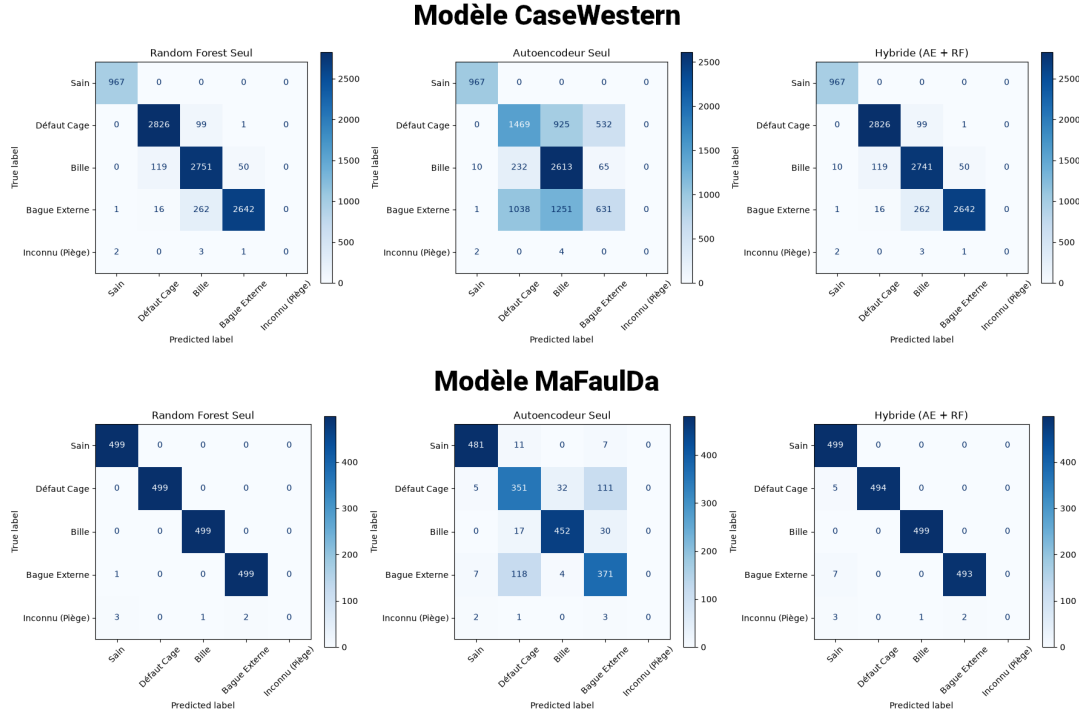


FIGURE 4.3 – Matrices de confusion de l'Architecture combinatoire proposée (AE + RF) évaluée sur les datasets CWRU et MaFaulDa.

4.3 Évaluation sur les Cas Extrêmes et Pièges

Afin d'attester de la validité concrète du système en conditions réelles et de garantir sa robustesse face à des événements industriels totalement imprévus ou aberrants, une large batterie de tests ciblés a été minutieusement conçue et ajoutée à la base de données de validation classique. En régime de croisière, le système classe avec un grand succès l'ensemble des états nominaux connus du dataset (Fonctionnement Sain, Défaut d'usure de la Cage, Défaut d'écrasement de la Bille, Défaut de frottement de la Bague Externe). De plus, l'architecture a été sévèrement éprouvée sur plusieurs cas de figures dits extrêmes ou de "corner cases" :

- La présentation d'une anomalie totalement inconnue du modèle d'entraînement, caractérisée artificiellement par un puissant pic de résonance pure à une fréquence aléatoire non caractéristique du roulement.
- L'injection d'un signal capteur affichant une valeur totalement plate (zéros continus), simulant fidèlement la défaillance matérielle ou la coupure physique du câble de l'accéléromètre.
- La simulation d'un choc mécanique isolé et très violent, constituant une panne extrême de grande amplitude hors des normes d'apprentissage.
- La conception d'une discordance spectrale artificielle, dont la forme d'onde globale du signal est mathématiquement aberrante et illogique par rapport au comportement mécanique physique attendu d'un équipement rotatif.
- L'émulation d'un défaut mécanique complexe et naissant, construit mathématiquement en associant et en pondérant le signal d'un roulement sain à seulement 5% de l'amplitude d'un signal de défaut avéré de bague externe, pour vérifier la sensibilité de détection précoce du système.

- Un scénario simulant empiriquement un défaut insidieux de lubrification lente (usure ou manque progressif de graisse industrielle) au moyen de l'injection continue d'un bruit de fond blanc très léger réparti sur l'ensemble du spectre.
- L'apparition soudaine d'un micro-pic de résonance aigu et ciblé sur une fréquence spécifique inattendue, souvent caractéristique d'une interférence électromagnétique ambiante dans l'usine.

TABLE 4.1 – Bilan détaillé et exhaustif des 11 tests extrêmes sur les architectures étudiées (Datasets CWRU et MaFaulDa)

N° & Scénario de Test	Résultat Attendu	Random Forest Seul	Autoencodeur Seul	combinatoire (AE + RF)
1. Moteur Sain (Nominal)	Maintien état Sain	Succès	Échec (Faux Positif)	Succès (Validé par l'AE)
2. Défaut Cage	Alerte : Défaut Cage	Succès	Succès	Succès
3. Défaut Bille	Alerte : Défaut Bille	Succès	Succès	Succès
4. Défaut Bague Externe	Alerte : Bague Externe	Succès	Échec Mauvais Diag. (CWRU) Succès (MaFaulDa)	Succès
5. Anomalie Inconnue	Blocage (Anomalie)	<i>Succès mitigé</i> (Forcé)*	Succès	<i>Succès mitigé</i> (Forcé)*
6. Signal Plat (Câble coupé)	Rejet pré-inférence	Bloqué	Bloqué	Bloqué
7. Panne Extrême (Choc)	Rejet pré-inférence	Bloqué	Bloqué	Bloqué
8. Discordance Spectrale	Blocage (Anomalie)	Succès (CWRU) Échec Classé Sain (MFD)	Succès	Succès (CWRU) Échec Faux Négatif (MFD)
9. Défaut Naissant (5% Ext.)	Alerte (Micro-défaut)	Échec (Classé Sain)	Succès	Échec (Faux Négatif)
10. Usure/Manque de graisse	Alerte (Micro-défaut)	Échec (Classé Sain)	Succès	Échec (Faux Négatif)
11. Micro-pic de résonance	Alerte (Micro-défaut)	Échec (Classé Sain)	Succès	Échec (Faux Négatif)

* Note : Bien que le système parvienne à lever une alerte sur l'anomalie inconnue, l'étape de classification effectuée en arrière-plan est une classification forcée totalement erronée (ex : classé arbitrairement "Bille" avec 45.8% de certitude sur CWRU). La règle de veto limitant la validation à 80% de certitude doit pallier ce phénomène en production.

L'analyse comparative de ces scénarios de crise (synthétisée dans le Tableau 4.1 compilant l'ensemble des résultats générés sur CWRU et MaFaulDa) met en lumière des comportements très contrastés selon l'architecture. Le système d'Autoencodeur (utilisé seul) détecte impitoyablement les anomalies, bloquant les résonances, les discordances et les micro-défauts naissants. Néanmoins, son exigence est telle qu'il échoue de manière critique sur le signal nominal du moteur sain (Test 1) en déclenchant de fausses alarmes, et peine parfois à qualifier correctement la panne (par exemple, une erreur de diagnostic de la bague externe confondue avec une bille sur CWRU).

Afin de pallier ces faux positifs inacceptables en milieu industriel, l'architecture combinatoire (AE + RF) s'appuie sur une hausse mathématique de son seuil d'erreur, mais repose surtout sur la capacité du Random Forest à venir corriger les erreurs de reconnaissance de l'Autoencodeur. Au cours de nos travaux, une hypothèse alternative avait d'abord été testée, dans laquelle le Random Forest n'avait jamais le droit de contredire l'alerte de l'Autoencodeur. Cependant, face au taux de fausses alarmes généré par l'AE sur les données saines, accorder un droit de correction au Random Forest s'est avéré indispensable. Dès lors, une tendance claire se dessine à la lecture des résultats : l'architecture combinatoire finit par calquer étroitement ses performances sur celles du Random Forest seul. Si cette dynamique de correction sécurise le système contre les fausses alertes sur l'état nominal, elle s'avère fortement pénalisante face aux dégradations extrêmement subtiles (défaut naissant à 5%, légère usure de graisse, micro-pic). Sur ces tests complexes, l'Autoencodeur parvient à percevoir l'anomalie, mais le Random Forest la corrige à tort en la requalifiant de "Saine", créant ainsi des faux négatifs (angles morts). Ce phénomène de nivellement par le bas est particulièrement notable sur le dataset très bruité de MaFaulDa, où la discordance spectrale (Test 8) parvient à tromper l'architecture combinatoire via cette même correction abusive du RF. Ces résultats soulignent que si l'architecture combinatoire sécurise magistralement le système contre les pannes majeures ou les alertes

intempestives, sa dépendance aux corrections du Random Forest lui fait perdre la sensibilité précoce de l'Autoencodeur.

4.4 Performances et Empreinte Matérielle

Les tableaux de métriques détaillés ci-dessous synthétisent de manière exhaustive les performances de classification statistiques et les temps d'exécution mesurés des différents modèles d'IA alors qu'ils sont exécutés en conditions réelles et chronométrées sur l'environnement matériel contraint de l'ESP32.

4.4.1 Résultats

Évaluée méthodiquement sur les différents sous-ensembles des jeux de données de référence, l'approche logicielle combinée (**AE + RF**) a vu ses performances temporelles et statistiques mesurées à des fins de comparaison rigoureuse. Ces tests ont été menés à la fois sur un ordinateur PC de référence classique (embarquant un CPU Intel Core i7-7500U, équipé de 8 GB de RAM et tournant sous environnement Linux Mint) et sur l'environnement très restreint du microcontrôleur ESP32 ciblé (voir Tableaux 4.2 et 4.3). Les modèles de bout en bout ont été intensivement entraînés en exploitant 60% de la totalité des échantillons vibratoires disponibles et l'accuracy du système a été validée de manière impartiale sur une portion de 20% des données totalement exclues de la phase d'apprentissage. Les temps d'inférence répertoriés dans les tableaux suivants représentent le cas d'usage le plus défavorable, c'est-à-dire le pire cas temporel possible (le traitement d'un signal qui requiert impérativement le déclenchement séquentiel et successif de l'Autoencodeur, puis des arbres du Random Forest pour un échantillon unique).

TABLE 4.2 – Performances Temporelles Comparatives - Dataset Standard Case Western (CWRU)

Modèle testé	Temps d'Entraînement Global	Inférence (PC Intel)	Inférence (Cible ESP32)	Accuracy
Random Forest (Supervisé)	30,96 s	6,68 ms	3,82 ms	94,60 %
Autoencodeur (Non-Supervisé)	42,58 s	58,09 ms	58,24 ms	58,50 %
Architecture Combinée (AE+RF)	73 s	63 ms	67,51 ms	94,53 %

TABLE 4.3 – Performances Temporelles Comparatives - Dataset Bruit et Complexe MaFaulDa

Modèle testé	Temps d'Entraînement Global	Inférence (PC Intel)	Inférence (Cible ESP32)	Accuracy
Random Forest (Supervisé)	6,21 s	7,36 ms	3,31 ms	99,80 %
Autoencodeur (Non-Supervisé)	27,23 s	71,09 ms	58,22 ms	70,64 %
Architecture Combinée (AE+RF)	39,15 s	74 ms	77,01 ms	99,00 %

L'analyse critique et objective de ces deux tableaux de métriques invite à une certaine nuance quant à la pertinence absolue de l'architecture combinatoire proposée, du moins lorsqu'elle est évaluée sur un spectre de données nominales. En effet, contrairement aux attentes initiales, l'architecture en deux étages ne surpasse pas le modèle supervisé isolé. Elle présente même une légère dégradation de la précision décisionnelle globale, passant de 94,60 % (pour le Random Forest seul) à 94,53 % sur le dataset de laboratoire CWRU, et de 99,80 % à 99,00 % sur le jeu d'enregistrements beaucoup plus complexe de MaFaulDa.

De surcroît, le coût temporel de cette sécurité supplémentaire pose un réel défi matériel. Le code en pur C++ du Random Forest transpilé s'affranchit de toute surcouche logicielle

et s'avère exceptionnellement vélocité sur la cible embarquée, ne nécessitant qu'un temps de calcul dérisoire de 3 à 4 millisecondes par échantillon. À l'inverse, l'obligation de faire transiter systématiquement et prioritairement chaque signal par le réseau de neurones de l'Autoencodeur fait exploser le temps d'inférence global de l'architecture combinée (atteignant 67 à 77 millisecondes sur l'ESP32). Par conséquent, au regard strict de ces résultats quantitatifs de base, l'architecture combinée (AE+RF) ne démontre pas de bénéfice direct et immédiat : elle est jusqu'à vingt fois plus lente que l'approche supervisée classique pour des résultats légèrement inférieurs. L'intérêt de ce pipeline ne se justifie donc pas par la maximisation des performances pures sur des pannes connues, mais doit être exclusivement vu comme une concession assumée : le sacrifice de la vitesse et d'une fraction de l'exactitude au profit d'un filet de sécurité théorique contre les anomalies inconnues ou aberrantes.

4.4.2 Performances théoriques

Pour permettre aux ingénieurs systèmes de mieux appréhender la faisabilité technique à long terme et l'évolutivité matérielle de cette implémentation logicielle spécifique sur un contrôleur de classe ESP32, le Tableau 4.4 ci-dessous synthétise et détaille la complexité intrinsèque de chacun des modèles mathématiques sous-jacents, quantifiant précisément le poids algorithmique à embarquer.

TABLE 4.4 – Caractéristiques Profilées des Modèles Embarqués

Nature du Modèle	Complexité Mathématique Formelle	Nombre de Paramètres	Volume d'Opérations Requis
Random Forest (Arbres)	$O(N_{trees} \times D)$	55 678 nœuds	804 Ops logiques
Autoencodeur (Réseau)	$O(W \times C)$	266 385 poids	573,4 kFLOPs
Arch. Combinée (AE+RF)	$O((W \times C) + (N_{trees} \times D))$	322 063 éléments	573,4 kFLOPs (+ 804 Ops)

** Notes techniques d'évaluation : N_{trees} représente le nombre d'arbres composant la forêt (80), D est la profondeur conditionnelle maximale de chaque arbre (10), W indique le nombre total de poids synaptiques quantifiés du réseau, et C modélise le coût computationnel relatif de calcul convolutif et matriciel par poids.*

Bien qu'il apparaisse que la volumineuse architecture globale proposée cumule logiquement en mémoire la totalité des paramètres individuels des deux modèles distincts qu'elle abrite (soit un total conséquent atteignant près de 322 063 paramètres à conserver en flash et à charger), son empreinte computationnelle et énergétique réelle sur le processeur Xtensa en cours d'inférence continue de rester très largement et logiquement dominée par la pléthore de multiplications matricielles nécessaires au traitement des couches denses et convolutives de l'Autoencodeur (totalisant plus de 573,4 kilo Opérations en Virgule Flottante par seconde). L'activation purement conditionnelle du traitement de l'étage 2 (le Random Forest transpilé en C++) n'ajoute en réalité à ce lourd bilan mathématique qu'un très faible et négligeable maximum absolu de 804 opérations logiques simples et ultra-optimisées par le compilateur matériel.

En guise de conclusion pragmatique sur ces tests en environnements réels, et en retenant avec la plus grande prudence les temps de cycle de calcul mesurés dans le scénario chronométré du pire des cas absolu (qui culminent à peine à 67,51 ms de traitement pour un bloc d'analyse issu du jeu CWRU et s'élèvent tout au plus à 77,01 ms pour un bloc plus bruité du contexte de MaFaulDa), force est de constater que la totalité et l'intégralité

du processus de diagnostic décisionnel de bout en bout parvient sans peine à s'exécuter confortablement en bien moins de la limite fatidique et symbolique des 100 millisecondes. Ce niveau de réactivité extrême garanti par le traitement Edge certifié de fait un système de surveillance doté d'un temps de réaction quasi-instantané à l'échelle des processus mécaniques lourds, autorisant amplement de déclencher un signal d'arrêt d'urgence sécurisé auprès de l'automate programmable d'une machine rotative bien en amont de l'apparition irréversible de vibrations critiques annonciatrices d'une rupture mécanique imminente de l'arbre de transmission.

Chapitre 5

Conclusion et Perspectives

5.1 Bilan des travaux

Au terme d'un processus de recherche et de développement itératif s'étant étalé sur l'intégralité de ce stage de fin d'études en ingénierie logicielle, nous avons formellement et empiriquement démontré avec un grand succès la viabilité technique et la robustesse de l'intégration poussée des technologies d'Intelligence Artificielle à la périphérie immédiate des systèmes (le paradigme de l'Edge AI) dans le but d'opérer une maintenance prédictive industrielle de haute fiabilité. L'utilisation stratégique d'un microcontrôleur commercial IoT particulièrement contraint en mémoire et en fréquence d'horloge tel que l'ESP32 ne constitue absolument pas un frein bloquant au traitement des algorithmes modernes d'apprentissage profond, à la condition impérieuse de repenser, d'adapter et d'optimiser rigoureusement chaque étape de la chaîne de traitement logicielle en tenant compte de la physique du processeur cible.

L'exploitation judicieuse des bibliothèques logicielles comme TFLite Micro (permettant d'embarquer l'inférence de l'Autoencodeur tout en conservant délibérément sa précision originelle en Float32 pour valider les capacités brutes du matériel), couplée à la création d'un algorithme de transpilation générant un code source C++ pur simulant les chemins décisionnels massifs d'un Random Forest, a permis de faire émerger un traitement combinatoire ambitieux. Cette architecture de décision séquentielle conçue sur mesure se révèle être exceptionnellement rapide d'exécution, hautement sécurisée par conception, et particulièrement économe en ressources énergétiques.

L'ambition théorique de cette architecture logicielle était d'allier la sécurité inhérente à une détection d'anomalies non supervisée (offrant un premier filtre universel) avec l'acuité catégorielle d'un système supervisé ciblé. Cependant, l'analyse critique et rigoureuse de nos résultats expérimentaux nous force à remettre en question la pertinence de cette architecture dans sa configuration actuelle. En effet, au sein de l'architecture combinatoire, l'Autoencodeur n'est entraîné qu'à détecter la normalité. Pour pallier sa sensibilité excessive (génératrice de faux positifs sur l'état sain), le système accorde au Random Forest le droit algorithmique de corriger et d'annuler les alertes de l'Autoencodeur. Ce compromis technique engendre un nivellement par le bas rédhibitoire : l'architecture hybride hérite directement des angles morts du modèle supervisé. Elle laisse passer les dégradations extrêmement précoces (micro-défauts) et se fait tromper par les mêmes pièges spectraux que le Random Forest seul, le veto de ce dernier annulant factuellement le bénéfice sécuritaire du filtre non-supervisé.

In fine, si cette approche combinatoire ne démontre pas de supériorité empirique justi-

fiant son déploiement en production sous cette configuration logique stricte — puisqu'elle délivre des performances statistiques quasi identiques au Random Forest isolé pour un temps d'inférence près de vingt fois supérieur —, ce constat d'échec algorithmique ne doit aucunement occulter la véritable réussite technique de l'expérience. En effet, ces travaux ont permis de prouver de manière irréfutable la faisabilité matérielle d'embarquer, de faire cohabiter et d'exécuter simultanément un réseau de neurones profond (Autoencodeur) et un modèle d'apprentissage ensembliste (Random Forest) sur un microcontrôleur aussi contraint que l'ESP32. Le fait que le système parvienne à enchaîner cette double inférence complexe en restant systématiquement sous la barre des 80 millisecondes, sans jamais provoquer de dépassement mémoire (Overflow), constitue un tour de force d'ingénierie logicielle (TinyML). Ce bilan offre donc une double conclusion scientifique majeure : d'une part, il valide pleinement le potentiel matériel de l'Edge AI pour supporter des architectures hybrides lourdes et asynchrones ; d'autre part, il pose une règle de conception fondamentale en prouvant qu'assujettir un détecteur d'anomalies aux corrections d'un classifieur supervisé détruit la détection de l'inconnu. L'ensemble de cette démarche exploratoire avait par ailleurs été pensé et dimensionné dès sa genèse pour tenter de répondre aux contraintes complexes des signaux de télémétrie du partenaire de recherche industriel Tractian.

5.2 Valorisation scientifique

Au-delà du succès du développement purement technique et logiciel détaillé dans ce rapport, une forte démarche institutionnelle de valorisation scientifique a été proactivement initiée pour encadrer et mettre en lumière ces travaux originaux de recherche appliquée. Dans ce contexte, un premier manuscrit d'article scientifique très détaillé a été minutieusement rédigé au cours du stage. Cet effort de rédaction approfondi avait pour finalité première de modéliser de manière très formelle la conception, le fonctionnement mathématique et les fondements théoriques de notre architecture de diagnostic vibratoire combinée à deux étages, et d'en exposer les métriques d'évaluation obtenues sur des cibles matérielles micro-embarquées. Bien que cet article novateur soit actuellement sous un statut de révision technique à usage interne aux équipes des universités impliquées (UBO et USP) et n'ait logiquement pas encore fait l'objet d'une soumission officielle devant un comité de lecture international, il constitue d'ores et déjà un socle textuel et méthodologique robuste, prêt à être affiné pour servir de fondation à de futures publications de plus grande envergure.

Par ailleurs, ces recherches novatrices ainsi que les résultats préliminaires très encourageants qu'elles ont engendrés ont déjà pu faire l'objet d'une première diffusion académique. Elles ont été formellement présentées de vive voix par mes soins, avec l'appui d'un poster scientifique de synthèse, lors de l'événement annuel de l'Assemblée Générale (AG) de l'équipe de recherche spécialisée SHARP au cœur du Lab-STICC de Brest. Cette excellente opportunité d'exposition locale a permis de stimuler la discussion, d'échanger des retours constructifs et de débattre de ces approches conceptuelles atypiques d'Edge AI avec la communauté des experts chercheurs du laboratoire régional.

5.3 Perspectives d'évolution

Indéniablement prometteurs dans le contexte actuel de transformation numérique mondiale, les résultats de ces vastes travaux méthodologiques posent des bases scientifiques très solides qui augurent d'excellentes perspectives pour de futurs développements concrets au sein du paradigme de l'Industrie 4.0. Dans l'optique de tendre vers un produit fini ou une architecture de recherche encore plus résiliente, plusieurs axes d'amélioration critiques peuvent aisément être envisagés à court et moyen terme :

- **Validation poussée in-situ** : Bien qu'intensivement éprouvé de manière virtuelle sur des bancs de tests mécaniques universitaires complexes et bruités (MaFaulDa), l'algorithme complet dans sa forme actuelle mérite grandement d'être prochainement déployé physiquement sur le terrain, en instrumentant en temps réel le nœud d'un capteur placé sur une chaîne de production active d'une grande usine. L'objectif sera d'évaluer de manière très fine son endurance et l'adaptabilité de ses modèles de détection face aux bruits parasites extrêmes de l'environnement industriel réel (vibrations de transmission issues des machines avoisinantes, transitoires électriques, variations thermiques).
- **Exploration et optimisation matérielle poussée** : Il devient crucial d'explorer massivement les possibilités offertes par la nouvelle génération de microcontrôleurs avancés intégrant des accélérateurs matériels appelés NPU (Neural Processing Unit), entièrement dédiés à l'accélération native des opérations tensorielles massivement parallèles. De plus, l'application future d'une véritable quantification en Int8 (Quantization) et de l'élagage des connexions (Pruning) permettrait d'exécuter des modèles génératifs bien plus profonds sans jamais saturer le processeur CPU principal de l'architecture.
- **Exploration d'une alternative algorithmique structurelle** : Au niveau purement logiciel, il serait particulièrement pertinent d'étudier en détail la faisabilité mathématique de la famille d'algorithmes des *Isolation Forests*. Cet algorithme de détection d'anomalies puissant et non conventionnel repose fondamentalement sur des structures internes d'ensembles d'arbres décisionnels. Il pourrait offrir les mêmes avantages inhérents à une détection non-supervisée universelle que l'Autoencodeur, tout en permettant de bénéficier intégralement de la légèreté structurelle de notre algorithme de conversion en langage C++ natif, réduisant ainsi par conception le besoin absolu de manipulations matricielles gourmandes en calculs.

Bibliographie

- [1] Cheng, W., et al. (2022) ‘Deep Robust Autoencoder based Framework for Bearing Fault Detection’, *IEEE International Conference on Prognostics and Health Management (PHM)*.
- [2] Smith, W.A. and Randall, R.B. (2015) ‘Rolling element bearing diagnostics using the Case Western Reserve University data : A benchmark study’, *Mechanical Systems and Signal Processing*.
- [3] Ibrahim, R., et al. (2024) ‘Fault Detection Based on Vibration Measurements and VAE’, *IEEE Open Journal of Industry Applications*.
- [4] Marques, C., et al. (2018) ‘Machinery Fault Database (MaFaulDa)’, *Universidade Federal do Rio de Janeiro*.

Annexe A

Article Scientifique

Le manuscrit d'article scientifique reproduit dans cette annexe a été intégralement rédigé au cours du stage en vue d'une future publication. Il synthétise et modélise formellement l'approche combinatoire développée, ainsi que les résultats expérimentaux obtenus sur la cible matérielle ESP32.

Autoencodeurs embarqués sur ESP32 pour la détection de vibrations

Alexandre Couté¹, Ana Fernanda Quaresma², Isadora Ferrão¹, Kalinka Branco², Catherine Dezan¹

¹ Université de Bretagne Occidentale, Lab-STICC

² Universidade de São Paulo, São Carlos

alexandre.coute@outlook.fr

Résumé

L'Industrie 4.0 déploie des modèles d'IA pour prédire les défaillances par analyse vibratoire. Cet article évalue deux modèles d'autoencodeur non-supervisé (Autoencodeur standard et VAE) pour la détection de vibration. Ces modèles sont évalués sur le Dataset CWRU et comparés avec un modèle supervisé de type Random Forest. Nous comparons leurs contraintes d'apprentissage, leur capacité de classification et leur portabilité sur ESP32, menant à la proposition d'une architecture optimisée pour la sûreté de fonctionnement.

Mots-clés

Edge AI, ESP32, Maintenance prédictive, Random Forest, Autoencodeur, TinyML, CWRU Dataset

Abstract

Industry 4.0 deploys AI models to predict failures through vibration analysis. This paper evaluates two unsupervised autoencoder models (standard Autoencoder and VAE) for vibration detection. These models are evaluated on the CWRU dataset and compared with a supervised Random Forest model. We compare their training constraints, classification ability, and portability on ESP32, leading to the proposal of an architecture optimized for functional safety.

1 Introduction et méthodologie

La maintenance prédictive industrielle requiert une analyse locale (Edge Computing) pour limiter la latence et sécuriser les données. Dans cette étude, on propose d'utiliser deux modèles non supervisés (Autoencodeur classique et Autoencodeur Variationnel) et de les comparer à un modèle supervisé (Random Forest).

Les données, issues de la Case Western Reserve University (CWRU) [1], ont été transformées par FFT (128 bandes) et normalisées. Afin de garantir la fiabilité statistique des modèles, l'entraînement utilise 80% des données disponibles, tandis que les 20% restants sont réservés à la validation et à l'évaluation des performances.

Pour garantir une comparaison non biaisée de leur capacité de séparation, l'étude se concentre sur l'état sain et trois défauts à une **sévérité unique de 0,007 pouce**. Ces défauts correspondent à des trous de 0,18 mm percé sur la machine

pour tester la capacité de l'IA à détecter l'apparition de premiers stades d'usure. Une méthode de *Data Augmentation* par injection de bruit gaussien permettra d'évaluer la robustesse de l'extraction des caractéristiques.

2 Description des modèles

Dans cette étude, trois modèles de type machine learning sont évalués :

- **Random Forest (RF)** : Un modèle d'apprentissage supervisé basé sur un ensemble d'arbres de décision. Sa structure logique le rend très léger, mais il requiert des données annotées pour l'ensemble des comportements (sains et défectueux).
- **Autoencodeur (AE) standard** : Un réseau de neurones non supervisé conçu pour compresser les données d'entrée dans un espace latent réduit, puis les reconstruire. L'objectif du modèle est de minimiser l'erreur quadratique moyenne (MSE) entre le signal d'entrée et le signal reconstruit [3].
- **Autoencodeur Variationnel (VAE)** : Une variante de l'autoencodeur qui cartographie les entrées selon une distribution probabiliste, permettant un espace latent plus continu mais avec une régularisation plus contraignante [2].

3 TinyML pour l'embarqué

La mise en oeuvre du modèle d'autoencodeur sur le microcontrôleur ESP32 (~520 Ko de RAM) nécessite un processus d'exportation et de conversion spécifique. Les modèles Keras traditionnels étant trop lourds pour l'ESP32, le modèle est compressé à l'aide de **TFLiteConverter** vers le format TensorFlow Lite, qui est léger et optimisé pour l'inférence. Afin d'éviter l'utilisation d'une carte SD externe, ce fichier binaire est traduit en un tableau C. Ce tableau est ainsi flashé directement dans la mémoire du microcontrôleur.

Dans l'environnement Arduino IDE, l'exploitation du modèle requiert l'installation de la bibliothèque **TFLite_ESP32**. L'architecture logicielle alloue manuellement un bloc de mémoire fixe dédié aux calculs de l'IA, appelé "Tensor Arena". Par ailleurs, les paramètres statistiques comme les minimums, maximums, et le seuil d'alerte calculé lors de l'entraînement sont intégrés comme

constantes dans le programme.

La boucle d'exécution logicielle s'organise en trois étapes :

- Le système capture des fenêtres de 256 points du signal et les transmet à l'entrée du modèle. L'autoencodeur passe les points dans son réseau de neurones pour générer le signal reconstruit.
- Le programme calcule ensuite en C++ la MSE mesurant la différence entre l'entrée et la reconstruction.
- Cette erreur est comparée au seuil d'alerte. Si la valeur calculée est excessivement haute par rapport au seuil, une anomalie est détectée, sinon l'état est considéré normal.

4 Expérimentations et résultats

L'évaluation repose sur l'accuracy, la classification (dont la classification forcée des signaux inconnus), la PCA, l'analyse des fréquences et l'empreinte matérielle. Le Tableau 1 synthétise ces performances.

TABLE 1 – Performances et contraintes matérielles des modèles

Critère	Random Forest	AE Standard	VAE
Accuracy (Sain + 0.007")	99,94%	88,5%	97,5%
Risque de classification forcée	100%	0%	0%
Temps Inférence (théorique)	< 1 ms	~ 30 ms	~ 50 ms
Mémoire RAM (ESP32)	< 1 Ko	~ 40 Ko	~ 60 Ko

Avec une **accuracy** de 99,94%, le **RF** domine la classification supervisée en isolant les pannes connues. L'**AE** (88,5%) et le **VAE** (97,5%) affichent une précision globale légèrement moindre car leur objectif n'étant pas de séparer des classes mais de modéliser le signal sain, ils sacrifient une fraction d'**accuracy** au profit d'une sécurité indispensable face à l'imprévu.

En milieu industriel, posséder des données étiquetées pour chaque dégradation est souvent irréaliste. C'est la vulnérabilité du **RF** (supervisé) : exigeant un dataset complet, il assigne tout signal inédit à une classe connue, générant un diagnostic erroné (**classification forcée**). À l'inverse, les **autoencodeurs** (non supervisés) s'entraînent uniquement sur l'état sain pour modéliser la norme. Tout signal déviant fait alors exploser la MSE, alertant l'opérateur. Cette flexibilité garantit un système robuste.

La **PCA** (Fig. 1) vérifie si le modèle structure logiquement les données en regroupant les états mécaniques similaires. Pour le **RF**, dépourvu d'espace latent, la projection des entrées brutes montre un chevauchement des pannes. L'**AE** projette efficacement les signaux en clusters géométriques isolés. Le **VAE** regroupe également les classes, mais sa contrainte probabiliste force un léger chevauchement.

L'analyse des **fréquences** (Fig. 2) révèle que **RF** base ses décisions en ciblant uniquement l'importance de pics fréquentiels précis du spectre FFT. À l'inverse, l'**AE** et le **VAE** analysent le spectre dans sa globalité pour reconstruire la

signature du signal sain. Face à une anomalie, c'est l'écart entre le spectre d'entrée et la reconstruction qui déclenche l'alerte.

La compression de l'information est le pilier de l'**embarcation sur ESP32**, rendant l'inférence des **autoencodeurs** gérable. Le **RF**, quant à lui, ne compresse pas les données mais compense par une conversion en règles C++ natives. Il offre ainsi une empreinte quasi nulle (< 1 Ko) et une latence minimale.

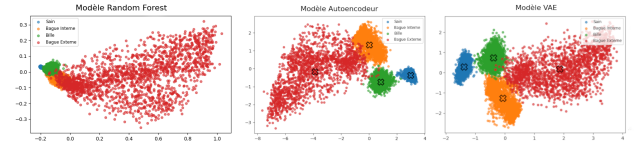


FIGURE 1 – Projection PCA des trois modèles.

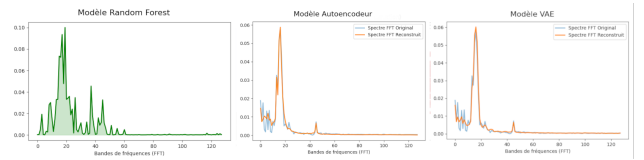


FIGURE 2 – Fréquences des trois modèles et reconstructions des autoencodeurs.

Le bilan de cette étude souligne le compromis entre **accuracy** et **généralisation**. Si le RF offre une exactitude sur les pannes apprises, les **autoencodeurs** délimitent strictement la normalité pour isoler toute défaillance inédite. C'est cette complémentarité qui justifie notre perspective d'architecture hybride : l'autoencodeur assure une détection universelle, tandis que le **RF** spécialise le diagnostic.

5 Conclusion et perspectives

Ce travail confirme la viabilité de l'IA embarquée pour la maintenance prédictive sur architectures contraintes. En perspective, nous proposons une solution hybride rapide : l'autoencodeur servira de détecteur universel, relayé par le Random Forest pour classer les défauts. Afin d'éviter toute classification forcée, un seuil de confiance de 80% isolera les pannes inconnues, garantissant ainsi un diagnostic sûr et exhaustif en conditions réelles.

Références

- [1] CWRU Bearing Data Center. Case Western Reserve University Dataset, <https://engineering.case.edu/bearingdatacenter>, 2026.
- [2] R. Ibrahim, et al. Fault Detection Based on Vibration Measurements and VAE. *IEEE OJIA*, 2024.
- [3] W. Cheng, et al. Deep Robust Autoencoder based Framework for Bearing Fault Detection. *IEEE PHM*, 2022.

Annexe B

Poster Scientifique

Le poster scientifique présenté ci-après a servi de support visuel lors de l'Assemblée Générale annuelle de l'équipe de recherche SHARP (Lab-STICC). Il offre une vulgarisation visuelle de l'architecture d'Edge AI appliquée à la maintenance prédictive.

Introduction

In the context of Industry 4.0, unexpected faults in rotating equipment regularly occur, causing critical production downtime. To ensure better productivity, this project aims to integrate Artificial Intelligence for the early diagnosis and prognosis of faults through the analysis of **vibration** data, facilitating a transition toward advanced **predictive maintenance** strategies.

Motivation:

- Unexpected production downtime represents a critical cost for manufacturers.
- Deploying Machine Learning models via *Edge Computing* (specifically **TinyML** on microcontrollers) allows data analysis right at the sensor level, thereby reducing latency, minimizing cloud dependency, and securing sensitive data.

Methodology: TinyML on ESP32

Embedded Processing Flow:

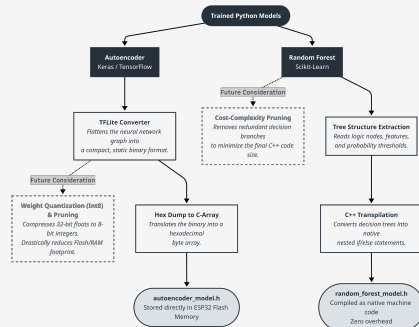


Fig. 3 — TinyML model optimization and deployment pipeline.

Transformations and Optimizations:

- **Implemented:** The Autoencoder is flattened into a static binary format and exported as a direct C-array via TFLite Micro. The Random Forest is transpiled into native C++ nested if/else rules, eliminating any framework overhead.
- **Transformations if needed:** **Weight Quantization (Int8)** for the Autoencoder (32-bit floats into 8-bit integers) and **Cost-Complexity Pruning** for the Random Forest to remove redundant tree branches.

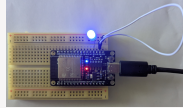


Fig. 4 — ESP32

ESP32 Constraints:
~520 KB SRAM, 4 MB Flash, 240 MHz CPU.

First Results (on healthy and faulty files)

Model Proposal ("Both"): Combining the Autoencoder to block unknown anomalies and the Random Forest to provide a precise diagnosis (identifying the specific defective part) once the anomaly is confirmed.

Table 1: Timing Performances - Case Western Dataset (CWRU)

Model	Train Time	Inf. PC	Inf. ESP32	Accuracy
Random Forest	30.96 s	6.68 ms	3.82 ms	94.60 %
Autoencoder	42.58 s	58.09 ms	58.24 ms	58.50 %
Both (AE+RF)	73 s	63 ms	67.51 ms	94.53 %

Table 2: Timing Performances - MaFaulDa Dataset

Model	Train Time	Inf. PC	Inf. ESP32	Accuracy
Random Forest	6.21 s	7.36 ms	3.31 ms	99.80 %
Autoencoder	27.23 s	71.09 ms	58.22 ms	70.64 %
Both (AE+RF)	39.15 s	74 ms	77.01 ms	99.00 %

* Notes: Training on 60% of samples, Accuracy on 20%. Inference times show the worst-case for a single sample.
Inf. PC : CPU Intel Core i7-7500U, 8 GB RAM (Linux Mint)

Table 3: Models Characteristics

Model	Complexity	Parameters	Operations
Random Forest	$O(N_{trees} \times D)$	55 678	804 Ops
Autoencoder	$O(W \times C)$	266 385	573.4 kFLOPs
Both (AE+RF)	Variable	322 063	573.4 kFLOPs (+ 804 Ops max)

* N_{trees} : Number of trees, D : Maximum tree depth, W : Number of weights, C : Computational cost per weight.

Context

Definitions:

- **Vibration:** Mechanical oscillation of a component around an equilibrium point. In industry, an alteration in the vibratory signature is the first physical symptom of hardware degradation.
- **Predictive Maintenance:** A strategy consisting of real-time equipment monitoring to predict and schedule interventions *before* critical failure occurs, thus optimizing machine lifespan.
- **Bearing:** An essential mechanical component that reduces friction between moving parts. It consists of four main parts prone to wear: the inner race, the outer race, the balls and the cage.

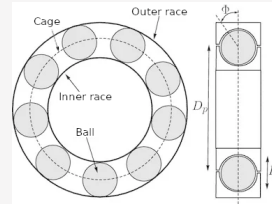


Fig. 1 — Internal structure of a ball bearing and potential failure zones

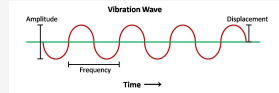


Fig. 2 — Explanation of a vibration

Experimentation

Studied Datasets:

- **Case Western (CWRU)**^[1]: Artificial lab faults (EDM) Distinct signatures, Easy detection.
- **MaFaulDa**^[2]: Mechanical simulator Complex intertwined wear, Subtle signatures, Realistic & Hard.

Supervised vs. Unsupervised:

- **Supervised (Random Forest):** Learns predefined classes only. Precise diagnosis for known conditions.
- **Unsupervised (Autoencoder)**^[3]: Learns to compress and reconstruct data. High error (MSE) detects unseen anomaly.

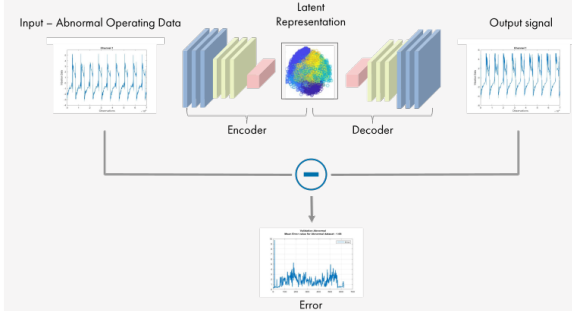


Fig. 5 — Architecture of an Autoencoder [4]

Conclusion & Perspectives

Conclusion:

- **TinyML Feasibility:** The ESP32 successfully runs complex ML pipelines (Autoencoder + Random Forest) locally without extreme model degradation.
- **Deployment Success:** TFLite Micro and native C++ transpilation enabled a seamless transition from Python to embedded hardware.
- **Performance:** Fast inference (~100 ms) and minimal memory footprint prove *Edge AI* is a viable, real-time solution for Industry 4.0.

Perspectives:

- Prove the viability of the model proposal by testing its resilience against previously unseen faults in real-world conditions.
- Maximize the optimization of the models implementation on the ESP32, specifically through advanced data transformations and hardware-specific model compression.
- Explore an unsupervised approach by implementing and evaluating an Isolation Forest solution for comparative anomaly detection.

References

- [1] CWRU Bearing Data Center. *Case Western Reserve University Dataset*, <https://engineering.case.edu/bearingdatacenter>, 2012.
- [2] Universidade Federal do Rio de Janeiro (UFRJ). *Machinery Fault Database (MaFaulDa)*, www02.smt.ufrj.br/~offshore/mfs.
- [3] R. Ibrahim et al., "Fault Detection Based on Vibration Measurements and VAE," *IEEE OJIA*, 2024.
- [4] MathWorks. *What Is an Autoencoder?*, fr.mathworks.com/discovery/autoencoder.html.